

SOLIDWORKS API (.NET VBA C#) — Examples (20/50): Expand_and_Collapse_FeatureManager_Desig- Fire_Notification_After_Adding_a_Mate_Ex

SOLIDWORKS API Documentation (converted from CHM)

Expand or Collapse FeatureManager Design Tree Nodes Example (VB.NET)

This example shows how to traverse, expand, and collapse the nodes of a FeatureManager design tree.

```
' -----  
' Preconditions:  
' 1. Open a part or assembly document.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Expands all of the FeatureManager design tree nodes.  
' 2. Click OK to collapse all nodes.  
' 3. Inspect the Immediate window.  
' -----  
  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Dim traverseLevel As Integer  
    Dim expandThis As Boolean  
  
    Sub main()  
        Dim i As Integer  
  
        Dim myModel As ModelDoc2  
        Dim featureMgr As FeatureManager  
        Dim rootNode As TreeControlItem  
  
        myModel = swApp.ActiveDoc  
        featureMgr = myModel.FeatureManager  
        rootNode = featureMgr.GetFeatureTreeRootItem2(swFeatMgrPane_e.swFeatMgrPaneBottom)  
  
        expandThis = True  
  
        For i = 0 To 1  
            If Not rootNode Is Nothing Then  
                Debug.Print("")  
                traverseLevel = 0  
                traverse_node(rootNode)  
            End If  
  
            expandThis = False  
  
            If i = 0 Then  
                MsgBox("OK to collapse all nodes?")  
            End If  
        Next  
    End Sub  
  
    Private Sub traverse_node(ByVal node As TreeControlItem)  
  
        Dim childNode As TreeControlItem  
        Dim featureNode As Feature  
        Dim componentNode As Component2  
        Dim nodeObjectType As Long  
        Dim nodeObject As Object  
        Dim restOfString As String = ""  
        Dim indent As String = ""  
        Dim i As Integer  
        Dim displayNodeInfo As Boolean  
        Dim compName As String  
        Dim suppr As Long, supprString As String = ""  
        Dim vis As Long, visString As String = ""  
        Dim fixed As Boolean, fixedString As String = ""
```

```

Dim componentDoc As Object, docString As String = ""
Dim refConfigName As String = ""

displayNodeInfo = False
nodeObjectType = node.ObjectType
nodeObject = node.Object

Select Case nodeObjectType
    Case swTreeControlItemType_e.swFeatureManagerItem_Feature

        displayNodeInfo = True
        If Not nodeObject Is Nothing Then
            featureNode = nodeObject
            restOfString = "[FEATURE: " & featureNode.Name & "]"
        Else
            restOfString = "[FEATURE: object Null?!]"
        End If

    Case swTreeControlItemType_e.swFeatureManagerItem_Component

        displayNodeInfo = True

        If Not nodeObject Is Nothing Then
            componentNode = nodeObject
            compName = componentNode.Name2

            If (compName = "") Then
                compName = "???"
            End If

            suppr = componentNode.GetSuppression

            Select Case (suppr)

                Case swComponentSuppressionState_e.swComponentFullyResolved
                    supprString = "Resolved"

                Case swComponentSuppressionState_e.swComponentLightweight
                    supprString = "Lightweight"

                Case swComponentSuppressionState_e.swComponentSuppressed
                    supprString = "Suppressed"

            End Select

            vis = componentNode.Visible

            Select Case (vis)

                Case swComponentVisibilityState_e.swComponentHidden
                    visString = "Hidden"

                Case swComponentVisibilityState_e.swComponentVisible
                    visString = "Visible"

            End Select

            fixed = componentNode.IsFixed

            If fixed = 0 Then
                fixedString = "Floating"
            Else
                fixedString = "Fixed"
            End If

            componentDoc = componentNode.GetModelDoc2

            If componentDoc Is Nothing Then
                docString = "NotLoaded"
            Else
                docString = "Loaded"
            End If

            refConfigName = componentNode.ReferencedConfiguration

```

```

        If (refConfigName = "") Then
            refConfigName = "???"
        End If

        restOfString = "[COMPONENT: " & compName & " " & docString & " " & supprString & "
" & visString & " " & refConfigName & "]"
    Else
        restOfString = "[COMPONENT: object Null?!]"
    End If

    Case Else

        displayNodeInfo = True

        If Not nodeObject Is Nothing Then
            restOfString = "[object type not handled]"
        Else
            restOfString = "[object Null?!]"
        End If

    End Select

    For i = 1 To traverseLevel
        indent = indent & "  "
    Next i

    If (displayNodeInfo) Then
        Debug.Print(indent & node.Text & " : " & restOfString)
    End If

    ' Expand the node
    node.Expanded = expandThis
    traverseLevel = traverseLevel + 1
    childNode = node.GetFirstChild

    While Not childNode Is Nothing
        Debug.Print(indent & "Node is expanded: " & childNode.Expanded)
        traverse_node(childNode)
        childNode = childNode.GetNext
    End While

    traverseLevel = traverseLevel - 1

End Sub

Public swApp As SldWorks

End Class

```

Expand or Collapse FeatureManager Design Tree Nodes Example (VBA)

This example shows how to traverse, expand, and collapse the nodes of a FeatureManager design tree.

```
'-----  
' Preconditions:  
' 1. Open a part or assembly document.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Expands all of the FeatureManager design tree nodes.  
' 2. Click OK to collapse all nodes.  
' 3. Inspect the Immediate window.  
'-----
```

Option Explicit

Dim traverseLevel As Integer

Dim expandThis As Boolean

Sub main()

Dim i As Integer

Dim swApp As SldWorks.SldWorks

Dim myModel As SldWorks.ModelDoc2

Dim featureMgr As SldWorks.FeatureManager

Dim rootNode As SldWorks.TreeControlItem

Set swApp = Application.SldWorks

Set myModel = swApp.ActiveDoc

Set featureMgr = myModel.FeatureManager

Set rootNode = featureMgr.GetFeatureTreeRootItem2(swFeatMgrPaneBottom)

expandThis = True

For i = 0 To 1

If Not rootNode Is Nothing Then

Debug.Print

traverseLevel = 0

traverse_node rootNode

End If

expandThis = False

If i = 0 Then

MsgBox "OK to collapse all nodes?"

End If

Next

End Sub

Private Sub traverse_node(node As SldWorks.TreeControlItem)

Dim childNode As SldWorks.TreeControlItem

Dim featureNode As SldWorks.Feature

Dim componentNode As SldWorks.Component2

Dim nodeObjectType As Long

Dim nodeObject As Object

Dim restOfString As String

Dim indent As String

Dim i As Integer

Dim displayNodeInfo As Boolean

Dim compName As String

Dim suppr As Long, supprString As String

Dim vis As Long, visString As String

Dim fixed As Boolean, fixedString As String

Dim componentDoc As Object, docString As String

Dim refConfigName As String

```

displayNodeInfo = False
nodeObjectType = node.ObjectType
Set nodeObject = node.Object
Select Case nodeObjectType

    Case SwConst.swTreeControlItemType_e.swFeatureManagerItem_Feature:

        displayNodeInfo = True
        If Not nodeObject Is Nothing Then
            Set featureNode = nodeObject
            restOfString = "[FEATURE: " & featureNode.Name & "]"
        Else
            restOfString = "[FEATURE: object Null?!]"
        End If

    Case SwConst.swTreeControlItemType_e.swFeatureManagerItem_Component:

        displayNodeInfo = True

        If Not nodeObject Is Nothing Then
            Set componentNode = nodeObject
            compName = componentNode.Name2

            If (compName = "") Then
                compName = "???"
            End If

            suppr = componentNode.GetSuppression

            Select Case (suppr)

                Case SwConst.swComponentSuppressionState_e.swComponentFullyResolved
                    supprString = "Resolved"

                Case SwConst.swComponentSuppressionState_e.swComponentLightweight
                    supprString = "Lightweight"

                Case SwConst.swComponentSuppressionState_e.swComponentSuppressed
                    supprString = "Suppressed"

            End Select

            vis = componentNode.Visible

            Select Case (vis)

                Case SwConst.swComponentVisibilityState_e.swComponentHidden
                    visString = "Hidden"

                Case SwConst.swComponentVisibilityState_e.swComponentVisible
                    visString = "Visible"

            End Select

            fixed = componentNode.IsFixed

            If fixed = 0 Then
                fixedString = "Floating"
            Else
                fixedString = "Fixed"
            End If

            Set componentDoc = componentNode.GetModelDoc2

```

```

        If componentDoc Is Nothing Then
            docString = "NotLoaded"
        Else
            docString = "Loaded"
        End If

        refConfigName = componentNode.ReferencedConfiguration

        If (refConfigName = "") Then
            refConfigName = "???"
        End If

        restOfString = "[COMPONENT: " & compName & " " & docString & " " & supprString & " " &
visString & " " & refConfigName & "]"
        Else
            restOfString = "[COMPONENT: object Null?!]"
        End If

        Case Else:
            displayNodeInfo = True
            If Not nodeObject Is Nothing Then
                restOfString = "[object type not handled]"
            Else
                restOfString = "[object Null?!]"
            End If
    End Select

    For i = 1 To traverseLevel
        indent = indent & " "
    Next i

    If (displayNodeInfo) Then
        Debug.Print indent & node.Text & " : " & restOfString
    End If

    ' Expand the node
    node.Expanded = expandThis
    traverseLevel = traverseLevel + 1
    Set childNode = node.GetFirstChild

    While Not childNode Is Nothing
        Debug.Print indent & "Node is expanded: " & childNode.Expanded
        traverse_node childNode
        Set childNode = childNode.GetNext
    Wend

    traverseLevel = traverseLevel - 1
End Sub

```

Expand Component in Specified FeatureManager Design Tree Pane Example (C#)

This example shows how to expand a component in the specified FeatureManager design tree pane.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\pdmworks\speaker.sldasm.
// 2. Select speaker_frame<1> in the FeatureManager design
//    tree.
//
// Postconditions:
// 1. Expands the selected component in the top pane of the
//    FeatureManager design tree.
// 2. Examine the top pane of the FeatureManager design tree
//    and the Immediate window.
//
// NOTE: Because this assembly is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssy = default(AssemblyDoc);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = default(Feature);
            Component2 swComp = default(Component2);
            FeatureManager swFeatMgr = default(FeatureManager);
            string featName = null;
            bool status = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssy = (AssemblyDoc)swModel;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swComp = (Component2)swSelMgr.GetSelectedObjectsComponent4(1, -1);
            swFeat = (Feature)swAssy.FeatureByName("speaker_frame-1");
            swFeat = (Feature)swComp.FeatureByName("Cut-Extrude1");
            featName = swFeat.Name;
            swFeatMgr = (FeatureManager)swModel.FeatureManager;
            status = swFeatMgr.ExpandFeature(swComp, featName, (int)swFeatMgrPane_e.swFeatMgrPaneTop);
            Debug.Print("Selected component expanded in top pane of FeatureManager design tree? " + status);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Expand Component in Specified FeatureManager Design Tree Pane Example (VBA)

This example shows how to expand a component in the specified FeatureManager design tree pane.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\pdmworks\speaker.sldasm.
' 2. Select speaker_frame<1> in the FeatureManager design
'    tree.
'
' Postconditions:
' 1. Expands the selected component in the top pane of the
'    FeatureManager design tree.
' 2. Examine the top pane of the FeatureManager design tree
'    and the Immediate window.
'
' NOTE: Because this assembly is used elsewhere, do not
' save changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swAssy As SldWorks.AssemblyDoc
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swComp As SldWorks.Component2
    Dim swFeatMgr As SldWorks.FeatureManager
    Dim featName As String
    Dim status As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssy = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swComp = swSelMgr.GetSelectedObjectsComponent4(1, -1)
    Set swFeat = swAssy.FeatureByName("speaker_frame-1")
    Set swFeat = swComp.FeatureByName("Cut-Extrude1")
    featName = swFeat.Name
    Set swFeatMgr = swModel.FeatureManager
    status = swFeatMgr.ExpandFeature(swComp, featName, swFeatMgrPane_e.swFeatMgrPaneTop)
    Debug.Print "Selected component expanded in top pane of FeatureManager design tree? " & status

End Sub
```

Expand Component in Specified FeatureManager Design Tree Pane Example (VB.NET)

This example shows how to expand a component in the specified FeatureManager design tree pane.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\pdmworks\speaker.sldasm.
' 2. Select speaker_frame<1> in the FeatureManager design
'    tree.
'
' Postconditions:
' 1. Expands the selected component in the top pane of the
'    FeatureManager design tree.
' 2. Examine the top pane of the FeatureManager design tree
'    and the Immediate window.
'
' NOTE: Because this assembly is used elsewhere, do not
'       save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swAssy As AssemblyDoc
        Dim swSelMgr As SelectionMgr
        Dim swFeat As Feature
        Dim swComp As Component2
        Dim swFeatMgr As FeatureManager
        Dim featName As String
        Dim status As Boolean

        swModel = swApp.ActiveDoc
        swAssy = swModel
        swSelMgr = swModel.SelectionManager
        swComp = swSelMgr.GetSelectedObjectsComponent4(1, -1)
        swFeat = swAssy.FeatureByName("speaker_frame-1")
        swFeat = swComp.FeatureByName("Cut-Extrude1")
        featName = swFeat.Name
        swFeatMgr = swModel.FeatureManager
        status = swFeatMgr.ExpandFeature(swComp, featName, swFeatMgrPane_e.swFeatMgrPaneTop)
        Debug.Print("Selected component expanded in top pane of FeatureManager design tree? " & status)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Export BOM's Second Column to BOM Table Area Example (C#)

This example shows how to export a BOM's second column to a BOM Table Area of a SOLIDWORKS MBD 3D PDF.

```
//-----
// Preconditions:
// 1. Verify that:
//    * specified assembly,
//    * specified SOLIDWORKS MBD 3D PDF theme, and
//    * c:\temp exist.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Inserts an indented BOM table in the assembly.
// 2. Gets the title of the second column in the BOM table
//    to export that column to the SOLIDWORKS MBD 3D PDF.
// 3. Gets the name of the BOM to map to the SOLIDWORKS
//    MBD 3D PDF.
// 4. Gets the SOLIDWORKS MBD 3D PDF data object.
//    a. Sets to display the SOLIDWORKS MBD 3D PDF after
//       publishing it.
//    b. Sets the path for the SOLIDWORKS MBD 3D PDF.
//    c. Sets the SOLIDWORKS MBD 3D PDF theme.
//    d. Sets the standard views for the SOLIDWORKS MBD 3D PDF.
//    e. Maps the BOM and exports its second column to a BOM
//       Table Area in the SOLIDWORKS MBD 3D PDF.
//    f. Publishes and displays the SOLIDWORKS MBD 3D PDF.
// 5. Examine c:\temp\MBDAssembly1.PDF and the Immediate window.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            BomTableAnnotation swBOMAnnotation = default(BomTableAnnotation);
            TableAnnotation swTableAnnotation = default(TableAnnotation);
            BomFeature swBOMFeature = default(BomFeature);
            MBD3DPdfData swMBDPdfData = default(MBD3DPdfData);
            string fileName = null;
            int errors = 0;
            int warnings = 0;
            int bomType = 0;
            string tableTemplate = null;
            string[] columnNames = new string[1];
            object columns = null;
            string BOMTableName = null;
            object standardViews = null;
            int[] viewIDs = new int[3];
            int nbrBOMTableAreas = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\advdrawings\\bladed shaft.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.
swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Insert indented BOM table in assembly
            bomType = (int)swBomType_e.swBomType_Indented;
            tableTemplate = "C:\\Program Files\\SOLIDWORKS Corp\\SOLIDWORKS\\lang\\english\\bom-standard.sldbomtbt";
            swBOMAnnotation = (BomTableAnnotation)swModelDocExt.InsertBomTable3(tableTemplate, 0, 1, bomType, "Default",
false, (int)swNumberingType_e.swNumberingType_Detailed, true);

            // Get title of second column in BOM table to export to SOLIDWORKS MBD 3D PDF
            swTableAnnotation = (TableAnnotation)swBOMAnnotation;
            columnNames[0] = swTableAnnotation.GetColumnTitle(1);
            Debug.Print("Title of second column to export to SOLIDWORKS MBD 3D PDF: " + columnNames[0]);
            columns = (object)columnNames;

            // Get name of BOM to map to SOLIDWORKS MBD 3D PDF
            swBOMFeature = (BomFeature)swBOMAnnotation.BomFeature;
            BOMTableName = swBOMFeature.Name;
            Debug.Print("Name of BOM to map to SOLIDWORKS MBD 3D PDF: " + BOMTableName);
        }
    }
}
```

```

// Get MBD3DPdfData object
swMBDPdfData = (MBD3DPdfData)swModelDocExt.GetMBD3DPdfData();

// Set to display SOLIDWORKS MBD 3D PDF
swMBDPdfData.ViewPdfAfterSaving = true;

// Set path for SOLIDWORKS MBD 3D PDF
swMBDPdfData.FilePath = "c:\\temp\\MBDAssembly1.PDF";

// Set SOLIDWORKS MBD 3D PDF theme
swMBDPdfData.ThemeName = "c:\\program files\\solidworks corp\\solidworks\\data\\themes\\simple assembly (a4,
landscape)\\theme.xml";

// Set standard views for SOLIDWORKS MBD 3D PDF
viewIDs[0] = (int)swStandardViews_e.swFrontView;
viewIDs[1] = (int)swStandardViews_e.swTopView;
viewIDs[2] = (int)swStandardViews_e.swDimetricView;
standardViews = (object)viewIDs;
swMBDPdfData.SetStandardViews(standardViews);

// Map BOM and export its second column to BOM Table Area
nbrBOMTableAreas = swMBDPdfData.GetBomAreaCount();
if (nbrBOMTableAreas > 0)
{
    swMBDPdfData.SetBomTable(0, BOMTableName, columns);
}

// Publish SOLIDWORKS MBD 3D PDF
swModelDocExt.PublishTo3DPDF(swMBDPdfData);
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Export BOM's Second Column to BOM Table Area Example (VBA)

This example shows how to export a BOM's second column to a BOM Table Area of a SOLIDWORKS MBD 3D PDF.

```
'-----
' Preconditions:
' 1. Verify that:
'   * specified assembly,
'   * specified SOLIDWORKS MBD 3D PDF theme, and
'   * c:\temp exist.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Inserts an indented BOM table in the assembly.
' 2. Gets the title of the second column in the BOM table
'   to export that column to the SOLIDWORKS MBD 3D PDF.
' 3. Gets the name of the BOM to map to the SOLIDWORKS
'   MBD 3D PDF.
' 4. Gets the SOLIDWORKS MBD 3D PDF data object.
'   a. Sets to display the SOLIDWORKS MBD 3D PDF after
'      publishing it.
'   b. Sets the path for the SOLIDWORKS MBD 3D PDF.
'   c. Sets the SOLIDWORKS MBD 3D PDF theme.
'   d. Sets the standard views for the SOLIDWORKS MBD 3D PDF.
'   e. Maps the BOM and exports its second column to a BOM
'      Table Area in the SOLIDWORKS MBD 3D PDF.
'   f. Publishes and displays the SOLIDWORKS MBD 3D PDF.
' 5. Examine c:\temp\MBDAssembly1.PDF and the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swBOMAnnotation As SldWorks.BomTableAnnotation
Dim swTableAnnotation As SldWorks.TableAnnotation
Dim swBOMFeature As SldWorks.BomFeature
Dim swMBDPdfData As SldWorks.MBD3DPdfData
Dim fileName As String
Dim errors As Long
Dim warnings As Long
Dim bomType As Long
Dim tableTemplate As String
Dim columnNames(0) As String
Dim columns As Variant
Dim BOMTableName As String
Dim standardViews As Variant
Dim viewIDs(2) As Long
Dim nbrBOMTableAreas As Long

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\bladed shaft.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent,
    "", errors, warnings)
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    ' Insert indented BOM table in assembly
    bomType = swBomType_Indented
    tableTemplate = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\bom-standard.sldbomtbt"
    Set swBOMAnnotation = swModelDocExt.InsertBomTable3(tableTemplate, 0, 1, bomType, "Default", False,
    swNumberingType_Detailed, True)

    ' Get title of second column in BOM table to export to SOLIDWORKS MBD 3D PDF
    Set swTableAnnotation = swBOMAnnotation
    columnNames(0) = swTableAnnotation.GetColumnTitle(1)
    Debug.Print "Title of second column to export to SOLIDWORKS MBD 3D PDF: " & columnNames(0)
    columns = columnNames
```

```

' Get name of BOM to map to SOLIDWORKS MBD 3D PDF
Set swBOMFeature = swBOMAnnotation.BomFeature
BOMTableName = swBOMFeature.Name
Debug.Print "Name of BOM to map to SOLIDWORKS MBD 3D PDF: " & BOMTableName

' Get MBD3PdfData object
Set swMBDPdfData = swModelDocExt.GetMBD3DPdfData

' Set to display SOLIDWORKS MBD 3D PDF
swMBDPdfData.ViewPdfAfterSaving = True

' Set path for SOLIDWORKS MBD 3D PDF
swMBDPdfData.FilePath = "c:\temp\MBDAssembly1.PDF"

' Set SOLIDWORKS MBD 3D PDF theme
swMBDPdfData.ThemeName = "c:\program files\solidworks corp\solidworks\data\themes\simple assembly (a4,
landscape)\theme.xml"

' Set standard views for SOLIDWORKS MBD 3D PDF
viewIDs(0) = swStandardViews_e.swFrontView
viewIDs(1) = swStandardViews_e.swTopView
viewIDs(2) = swStandardViews_e.swDimetricView
standardViews = viewIDs
swMBDPdfData.SetStandardViews (standardViews)

' Map BOM and export its second column to a BOM Table Area
nbrBOMTableAreas = swMBDPdfData.GetBomAreaCount
If nbrBOMTableAreas > 0 Then
    swMBDPdfData.SetBomTable 0, BOMTableName, columns
End If

' Publish SOLIDWORKS MBD 3D PDF
swModelDocExt.PublishTo3DPDF swMBDPdfData

End Sub

```

Export BOM's Second Column to BOM Table Area Example (VB.NET)

This example shows how to export a BOM's second column to a BOM Table Area of a SOLIDWORKS MBD 3D PDF.

```

'-----
' Preconditions:
' 1. Verify that:
'   * specified assembly,
'   * specified SOLIDWORKS MBD 3D PDF theme, and
'   * c:\temp exist.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Inserts an indented BOM table in the assembly.
' 2. Gets the title of the second column in the BOM table
'   to export that column to the SOLIDWORKS MBD 3D PDF.
' 3. Gets the name of the BOM to map to the SOLIDWORKS
'   MBD 3D PDF.
' 4. Gets the SOLIDWORKS MBD 3D PDF data object.
'   a. Sets to display the SOLIDWORKS MBD 3D PDF after
'      publishing it.
'   b. Sets the path for the SOLIDWORKS MBD 3D PDF.
'   c. Sets the SOLIDWORKS MBD 3D PDF theme.
'   d. Sets the standard views for the SOLIDWORKS MBD 3D PDF.
'   e. Maps the BOM and exports its second column to a BOM
'      Table Area in the SOLIDWORKS MBD 3D PDF.
'   f. Publishes and displays the SOLIDWORKS MBD 3D PDF.
' 5. Examine c:\temp\MBDAssembly1.PDF and the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swBOMAnnotation As BomTableAnnotation
        Dim swTableAnnotation As TableAnnotation
        Dim swBOMFeature As BomFeature
        Dim swMBDPdfData As MBD3DPdfData
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim bomType As Integer
        Dim tableTemplate As String
        Dim columnNames(0) As String
        Dim columns() As Object
        Dim BOMTableName As String
        Dim standardViews As Object
        Dim viewIDs(2) As Integer
        Dim nbrBOMTableAreas As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\
2018\samples\tutorial\advdrawings\bladed shaft.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent,
"", errors, warnings)
        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        ' Insert indented BOM table in assembly
        bomType = swBomType_e.swBomType_Indented
        tableTemplate = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\bom-standard.sldbomtbt"
        swBOMAnnotation = swModelDocExt.InsertBomTable3(tableTemplate, 0, 1, bomType, "Default", False, swNumberingType_
e.swNumberingType_Detailed, True)

        ' Get title of second column in BOM table to export to SOLIDWORKS MBD 3D PDF
        swTableAnnotation = swBOMAnnotation
        columnNames(0) = swTableAnnotation.GetColumnTitle(1)
        Debug.Print("Title of second column to export to SOLIDWORKS MBD 3D PDF: " & columnNames(0))
        columns = columnNames

        ' Get name of BOM to map to SOLIDWORKS MBD 3D PDF

```

```

swBOMFeature = swBOMAnnotation.BomFeature
BOMTableName = swBOMFeature.Name
Debug.Print("Name of BOM to map to SOLIDWORKS MBD 3D PDF: " & BOMTableName)

' Get MBD3DPdfData object
swMBDPdfData = swModelDocExt.GetMBD3DPdfData

' Set to display SOLIDWORKS MBD 3D PDF
swMBDPdfData.ViewPdfAfterSaving = True

' Set path for SOLIDWORKS MBD 3D PDF
swMBDPdfData.FilePath = "c:\temp\MBDAssembly1.PDF"

' Set SOLIDWORKS MBD 3D PDF theme
swMBDPdfData.ThemeName = "c:\program files\solidworks corp\solidworks\data\themes\simple assembly (a4, landscape
)\theme.xml"

' Set standard views for SOLIDWORKS MBD 3D PDF
viewIDs(0) = swStandardViews_e.swFrontView
viewIDs(1) = swStandardViews_e.swTopView
viewIDs(2) = swStandardViews_e.swDimetricView
standardViews = viewIDs
swMBDPdfData.SetStandardViews(standardViews)

' Map BOM and export its second column to a BOM Table Area
nbrBOMTableAreas = swMBDPdfData.GetBomAreaCount
If nbrBOMTableAreas > 0 Then
    swMBDPdfData.SetBomTable(0, BOMTableName, columns)
End If

' Publish SOLIDWORKS MBD 3D PDF
swModelDocExt.PublishTo3DPDF(swMBDPdfData)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Export BOMs to XML Example (C#)

SOLIDWORKS Design manages Bills of Materials (BOM) and controls the information within them. You can extract this information for use in downstream systems such as ERP or other business systems.

In SOLIDWORKS 2004 and later, BOMs are now features and appear during a traversal of the FeatureManager design tree. This example shows how to get to each BOM in a drawing document and save the BOM information to an XML file. You can transform this XML file using XSL or transfer the file to other systems.

```
//-----
// Preconditions:
// 1. Open a drawing document with at least on BOM.
// 2. Add a reference to Microsoft Scripting Runtime (right-click
//    the name of the project in the Project Explorer and click Add Reference >
//    the Browse tab > C:\windows\system32\scrrun.dll > OK.
//
// Postconditions:
// 1. Saves an XML file to the folder where the drawing document resides,
//    overwriting any existing file of the same name.
// 2. Examines the folder where the drawing document resides.
//
// NOTES:
// * XML tags are based on BOM column headings.
// * Invalid characters must be removed from the
//   column headings.
// * XML schema is:
//
//    <BOMS>
//      <SHEET>
//        <NAME>Sheet1</NAME>
//        <BOM>
//          <NAME>Bill Of Materials1</NAME>
//          <TABLE>
//            <ROW>
//              <ITEM_NO>1</ITEM_NO>
//              <PART_NUMBER>2004_Part</PART_NUMBER>
//              <DESCRIPTION></DESCRIPTION>
//              <QTY>2</QTY>
//            </ROW>
//            <ROW>
//              <ITEM_NO>2</ITEM_NO>
//              <PART_NUMBER>2004_Part</PART_NUMBER>
//              <DESCRIPTION></DESCRIPTION>
//              <QTY>1</QTY>
//            </ROW>
//            <ROW>
//              <ITEM_NO>3</ITEM_NO>
//              <PART_NUMBER>2004_Part</PART_NUMBER>
//              <DESCRIPTION></DESCRIPTION>
//              <QTY>1</QTY>
//            </ROW>
//            <ROW>
//              <ITEM_NO>4</ITEM_NO>
//              <PART_NUMBER>bead7</PART_NUMBER>
//              <DESCRIPTION></DESCRIPTION>
//              <QTY>1</QTY>
//            </ROW>
//          </TABLE>
//        </BOM>
//      </SHEET>
//    <SHEET>
//      <NAME>Sheet2</NAME>
//      <BOM>
//        <NAME>Bill Of Materials2</NAME>
//        <TABLE>
//          <ROW>
//            <ITEM_NO>1</ITEM_NO>
//            <PART_NUMBER>Assem3</PART_NUMBER>
//            <DESCRIPTION></DESCRIPTION>
//            <QTY>1</QTY>
//          </ROW>
//          <ROW>
//            <ITEM_NO></ITEM_NO>
//            <PART_NUMBER> cylinder</PART_NUMBER>
//            <DESCRIPTION></DESCRIPTION>
//            <QTY>1</QTY>
//          </ROW>
//          <ROW>
//            <ITEM_NO></ITEM_NO>
//            <PART_NUMBER> cylinder</PART_NUMBER>
//            <DESCRIPTION></DESCRIPTION>
//            <QTY>1</QTY>
//          </ROW>
//          <ROW>
//            <ITEM_NO></ITEM_NO>
//            <PART_NUMBER> SimpleCube_A</PART_NUMBER>
```

```

//      <DESCRIPTION></DESCRIPTION>
//      <QTY>1</QTY>
//    </ROW>
//    <ROW>
//      <ITEM_NO></ITEM_NO>
//      <PART_NUMBER> JoinedCyl</PART_NUMBER>
//      <DESCRIPTION></DESCRIPTION>
//      <QTY>1</QTY>
//    </ROW>
//  </TABLE>
// </BOM>
// <BOM>
//   <NAME>Bill Of Materials3</NAME>
//   <TABLE>
//     <ROW>
//       <ITEM_NO>8</ITEM_NO>
//       <PART_NUMBER>2004_Part</PART_NUMBER>
//       <DESCRIPTION></DESCRIPTION>
//       <QTY>2</QTY>
//     </ROW>
//   </TABLE>
//   <TABLE>
//     <ROW>
//       <ITEM_NO>9</ITEM_NO>
//       <PART_NUMBER>2004_Part</PART_NUMBER>
//       <DESCRIPTION></DESCRIPTION>
//       <QTY>1</QTY>
//     </ROW>
//     <ROW>
//       <ITEM_NO>10</ITEM_NO>
//       <PART_NUMBER>2004_Part</PART_NUMBER>
//       <DESCRIPTION></DESCRIPTION>
//       <QTY>1</QTY>
//     </ROW>
//   </TABLE>
//   <TABLE>
//     <ROW>
//       <ITEM_NO>11</ITEM_NO>
//       <PART_NUMBER>bead7</PART_NUMBER>
//       <DESCRIPTION></DESCRIPTION>
//       <QTY>1</QTY>
//     </ROW>
//   </TABLE>
// </BOM>
// <BOM>
//   <NAME>Bill Of Materials4</NAME>
//   <TABLE>
//     <TITLE>BOM Table 2</TITLE>
//     <ROW>
//       <ITEM_NO>1</ITEM_NO>
//       <PART_NUMBER>cylinder</PART_NUMBER>
//       <DESCRIPTION></DESCRIPTION>
//       <QTY>1</QTY>
//     </ROW>
//     <ROW>
//       <ITEM_NO>2</ITEM_NO>
//       <PART_NUMBER>cylinder</PART_NUMBER>
//       <DESCRIPTION></DESCRIPTION>
//       <QTY>1</QTY>
//     </ROW>
//   </TABLE>
//   <TABLE>
//     <TITLE>BOM Table 2</TITLE>
//     <ROW>
//       <ITEM_NO>3</ITEM_NO>
//       <PART_NUMBER>SimpleCube_A</PART_NUMBER>
//       <DESCRIPTION></DESCRIPTION>
//       <QTY>1</QTY>
//     </ROW>
//     <ROW>
//       <ITEM_NO>4</ITEM_NO>
//       <PART_NUMBER>JoinedCyl</PART_NUMBER>
//       <DESCRIPTION></DESCRIPTION>
//       <QTY>1</QTY>
//     </ROW>
//   </TABLE>
// </BOM>
// </SHEET>
// </BOMS>

```

```

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using Scripting;
using System.Diagnostics;

namespace XMLCsharp.csproj
{
    public partial class SolidWorksMacro
    {

```

```

public void Main()
{
    ModelDoc2 swModel = default(ModelDoc2);
    DrawingDoc swDraw = default(DrawingDoc);
    Feature swFeat = default(Feature);
    BomFeature swBomFeat = default(BomFeature);
    string sPathName = null;
    bool bIsFirstSheet = false;
    Scripting.FileSystemObject fso = default(Scripting.FileSystemObject);
    Scripting.TextStream XMLfile = default(Scripting.TextStream);

    swModel = (ModelDoc2)swApp.ActiveDoc;
    swDraw = (DrawingDoc)swModel;
    bIsFirstSheet = true;
    // Strip off SOLIDWORKS file extension (sldrw)
    // and add XML extension (xml)
    sPathName = swModel.GetPathName();
    //sPathName = Strings.Left(sPathName, Strings.Len(sPathName) - 6);
    sPathName = sPathName.Substring(0, sPathName.Length - 6);
    sPathName = sPathName + ".xml";
    //fso = Interaction.CreateObject("Scripting.FileSystemObject");
    fso = new Scripting.FileSystemObject();
    XMLfile = fso.CreateTextFile(sPathName, true, true);
    XMLfile.WriteLine("<BOMS>");
    swFeat = (Feature)swModel.FirstFeature();
    while ((swFeat != null))
    {
        if ("DrSheet" == swFeat.GetTypeName())
        {
            XMLfile.WriteLine("    <SHEET>");
            XMLfile.WriteLine("        <NAME>" + swFeat.Name + "</NAME>");
            bIsFirstSheet = false;
        }
        if ("BomFeat" == swFeat.GetTypeName())
        {
            swBomFeat = (BomFeature)swFeat.GetSpecificFeature2();
            ProcessBomFeature(swApp, swModel, swBomFeat, XMLfile);
        }
        swFeat = (Feature)swFeat.GetNextFeature();
        if ((swFeat != null))
        {
            if ("DrSheet" == swFeat.GetTypeName() & !bIsFirstSheet)
            {
                XMLfile.WriteLine("    </SHEET>");
            }
        }
    }

    XMLfile.WriteLine("    </SHEET>");
    XMLfile.WriteLine("</BOMS>");
    XMLfile.Close();
}

XMLfile)
public void ProcessTableAnn(SldWorks swApp, ModelDoc2 swModel, TableAnnotation swTableAnn, Scripting.TextStream
{
    int nNumRow = 0;
    int nNumCol = 0;
    int nNumHeader = 0;
    string[] sHeaderText = null;
    int j = 0;
    int k = 0;
    int nIndex = 0;
    int nCount = 0;
    int nStart = 0;
    int nEnd = 0;
    int nSplitDir = 0;

    nNumHeader = swTableAnn.GetHeaderCount();
    Debug.Assert(nNumHeader >= 1);
    nSplitDir = swTableAnn.GetSplitInformation(ref nIndex, ref nCount, ref nStart, ref nEnd);
    if ((int)swTableSplitDirection_e.swTableSplit_None == nSplitDir)
    {
        Debug.Assert(0 == nIndex);
        Debug.Assert(0 == nCount);
        Debug.Assert(0 == nStart);
        Debug.Assert(0 == nEnd);
        nNumRow = swTableAnn.RowCount;
        nNumCol = swTableAnn.ColumnCount;
        nStart = nNumHeader;
        nEnd = nNumRow - 1;
    }
    else
    {
        Debug.Assert((int)swTableSplitDirection_e.swTableSplit_Horizontal == nSplitDir);
        Debug.Assert(nIndex >= 0);
        Debug.Assert(nCount >= 0);
        Debug.Assert(nStart >= 0);
        Debug.Assert(nEnd >= nStart);
        nNumCol = swTableAnn.ColumnCount;
        if (1 == nIndex)
        {

```

```

        // Add header offset for first portion of table
        nStart = nStart + nNumHeader;
    }
}
XMLfile.WriteLine("                <TABLE>");
if (swTableAnn.TitleVisible)
{
    XMLfile.WriteLine("                <TITLE>" + swTableAnn.Title + "</TITLE>");
}
sHeaderText = new string[nNumCol];
for (j = 0; j <= nNumCol - 1; j++)
{
    sHeaderText[j] = (String)swTableAnn.GetColumnTitle2(j, true);
    // Replace invalid characters for XML tags
    sHeaderText[j] = sHeaderText[j].Replace(".", "");
    sHeaderText[j] = sHeaderText[j].Replace(" ", "_");
}
for (j = nStart; j <= nEnd; j++)
{
    XMLfile.WriteLine("                <ROW>");
    for (k = 0; k <= nNumCol - 1; k++)
    {
        XMLfile.WriteLine("                " + "<" + sHeaderText[k] + ">" + swTableAnn.get_Text2(j, k,
true) + "</" + sHeaderText[k] + ">");
    }
    XMLfile.WriteLine("                </ROW>");
}
XMLfile.WriteLine("            </TABLE>");
}
public void ProcessBomFeature(SldWorks swApp, ModelDoc2 swModel, BomFeature swBomFeat, Scripting.TextStream XMLf
ile)
{
    Feature swFeat = default(Feature);
    object[] vTableArr = null;
    object vTable = null;
    TableAnnotation swTable = default(TableAnnotation);

    swFeat = (Feature)swBomFeat.GetFeature();
    XMLfile.WriteLine("            <BOM>");
    XMLfile.WriteLine("            <NAME>" + swFeat.Name + "</NAME>");
    vTableArr = (object[])swBomFeat.GetTableAnnotations();
    foreach (object vTable_loopVariable in vTableArr)
    {
        vTable = vTable_loopVariable;
        swTable = (TableAnnotation)vTable;
        ProcessTableAnn(swApp, swModel, swTable, XMLfile);
    }
    XMLfile.WriteLine("            </BOM>");
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Export BOMs to XML Example (VBA)

SOLIDWORKS Design manages Bills of Materials (BOM) and controls the information within them. You can extract this information for use in downstream systems such as ERP or other business systems.

In SOLIDWORKS 2004 and later, BOMs are now features and appear during a traversal of the FeatureManager design tree. This example shows how to get to each BOM in a drawing document and save the BOM information to an XML file. You can transform this XML file using XSL or transfer the file to other systems.

```
'-----
' Preconditions:
' 1. Open a drawing document with at least on BOM.
' 2. Add a reference to Microsoft Scripting Runtime (click
'    Tools > References > Browse > C:\windows\system32\scrrun.dll.
'
' Postconditions:
' 1. Saves an XML file to the folder where the drawing document resides,
'    overwriting any existing file of the same name.
' 2. Examine the folder where the drawing document resides.
'
' NOTES:
' * XML tags are based on BOM column headings.
' * Invalid characters must be removed from the
'   column headings.
' * XML schema is:
'   <BOMS>
'     <SHEET>
'       <NAME>Sheet1</NAME>
'     <BOM>
'       <NAME>Bill Of Materials1</NAME>
'       <TABLE>
'         <ROW>
'           <ITEM_NO>1</ITEM_NO>
'           <PART_NUMBER>2004_Part</PART_NUMBER>
'           <DESCRIPTION></DESCRIPTION>
'           <QTY>2</QTY>
'         </ROW>
'         <ROW>
'           <ITEM_NO>2</ITEM_NO>
'           <PART_NUMBER>2004_Part</PART_NUMBER>
'           <DESCRIPTION></DESCRIPTION>
'           <QTY>1</QTY>
'         </ROW>
'         <ROW>
'           <ITEM_NO>3</ITEM_NO>
'           <PART_NUMBER>2004_Part</PART_NUMBER>
'           <DESCRIPTION></DESCRIPTION>
'           <QTY>1</QTY>
'         </ROW>
'         <ROW>
'           <ITEM_NO>4</ITEM_NO>
'           <PART_NUMBER>bead7</PART_NUMBER>
'           <DESCRIPTION></DESCRIPTION>
'           <QTY>1</QTY>
'         </ROW>
'       </TABLE>
'     </BOM>
'   </SHEET>
'   <SHEET>
'     <NAME>Sheet2</NAME>
'     <BOM>
'       <NAME>Bill Of Materials2</NAME>
'       <TABLE>
'         <ROW>
'           <ITEM_NO>1</ITEM_NO>
'           <PART_NUMBER>Assem3</PART_NUMBER>
'           <DESCRIPTION></DESCRIPTION>
'           <QTY>1</QTY>
'         </ROW>
'         <ROW>
'           <ITEM_NO></ITEM_NO>
'           <PART_NUMBER> cylinder</PART_NUMBER>
'           <DESCRIPTION></DESCRIPTION>
'           <QTY>1</QTY>
```

```

        </ROW>
        <ROW>
            <ITEM_NO></ITEM_NO>
            <PART_NUMBER> cylinder</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>1</QTY>
        </ROW>
        <ROW>
            <ITEM_NO></ITEM_NO>
            <PART_NUMBER> SimpleCube_A</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>1</QTY>
        </ROW>
        <ROW>
            <ITEM_NO></ITEM_NO>
            <PART_NUMBER> JoinedCyl</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>1</QTY>
        </ROW>
    </TABLE>
</BOM>
<BOM>
    <NAME>Bill Of Materials3</NAME>
    <TABLE>
        <ROW>
            <ITEM_NO>8</ITEM_NO>
            <PART_NUMBER>2004_Part</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>2</QTY>
        </ROW>
    </TABLE>
    <TABLE>
        <ROW>
            <ITEM_NO>9</ITEM_NO>
            <PART_NUMBER>2004_Part</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>1</QTY>
        </ROW>
        <ROW>
            <ITEM_NO>10</ITEM_NO>
            <PART_NUMBER>2004_Part</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>1</QTY>
        </ROW>
    </TABLE>
    <TABLE>
        <ROW>
            <ITEM_NO>11</ITEM_NO>
            <PART_NUMBER>bead7</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>1</QTY>
        </ROW>
    </TABLE>
</BOM>
<BOM>
    <NAME>Bill Of Materials4</NAME>
    <TABLE>
        <TITLE>BOM Table 2</TITLE>
        <ROW>
            <ITEM_NO>1</ITEM_NO>
            <PART_NUMBER>cylinder</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>1</QTY>
        </ROW>
        <ROW>
            <ITEM_NO>2</ITEM_NO>
            <PART_NUMBER>cylinder</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>1</QTY>
        </ROW>
    </TABLE>
    <TABLE>
        <TITLE>BOM Table 2</TITLE>
        <ROW>
            <ITEM_NO>3</ITEM_NO>
            <PART_NUMBER>SimpleCube_A</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>1</QTY>
        </ROW>
        <ROW>

```

```

'                                     <ITEM_NO>4</ITEM_NO>
'                                     <PART_NUMBER>JoinedCyl</PART_NUMBER>
'                                     <DESCRIPTION></DESCRIPTION>
'                                     <QTY>1</QTY>
'                                     </ROW>
'                               </TABLE>
'                         </BOM>
'                   </SHEET>
' </BOMS>
-----

```

Option Explicit

```

Sub ProcessTableAnn(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swTableAnn As SldWorks.TableAnnotation,
XMLfile As Scripting.TextStream)

```

```

    Dim numRows As Long
    Dim numCol As Long
    Dim numHeader As Long
    Dim sHeaderText() As String
    Dim j As Long
    Dim k As Long
    Dim nIndex As Long
    Dim nCount As Long
    Dim nStart As Long
    Dim nEnd As Long
    Dim nSplitDir As Long

    numRows = swTableAnn.GetHeaderCount: Debug.Assert numRows >= 1

    nSplitDir = swTableAnn.GetSplitInformation(nIndex, nCount, nStart, nEnd)

    If swTableSplit_None = nSplitDir Then
        Debug.Assert 0 = nIndex
        Debug.Assert 0 = nCount
        Debug.Assert 0 = nStart
        Debug.Assert 0 = nEnd
        numRows = swTableAnn.RowCount
        numCol = swTableAnn.ColumnCount
        nStart = numRows
        nEnd = numRows - 1
    Else
        Debug.Assert swTableSplit_Horizontal = nSplitDir
        Debug.Assert nIndex >= 0
        Debug.Assert nCount >= 0
        Debug.Assert nStart >= 0
        Debug.Assert nEnd >= nStart
        numCol = swTableAnn.ColumnCount
        If 1 = nIndex Then
            ' Add header offset for first portion of table
            nStart = nStart + numRows
        End If
    End If

    XMLfile.WriteLine "          <TABLE>"

    If swTableAnn.TitleVisible Then
        XMLfile.WriteLine "          <TITLE>" & swTableAnn.Title & "</TITLE>"
    End If

    ReDim sHeaderText(numCol - 1)

    For j = 0 To numCol - 1
        sHeaderText(j) = swTableAnn.GetColumnTitle2(j, True)
        ' Replace invalid characters for XML tags
        sHeaderText(j) = Replace(sHeaderText(j), ".", "")
        sHeaderText(j) = Replace(sHeaderText(j), " ", "_")
    Next j

    For j = nStart To nEnd
        XMLfile.WriteLine "          <ROW>"
        For k = 0 To numCol - 1
            XMLfile.WriteLine "          " & "<" & sHeaderText(k) & ">" & swTableAnn.Text2(j, k, True) & "</"
+ sHeaderText(k) & ">"
        Next k

        XMLfile.WriteLine "          </ROW>"
    Next j

    XMLfile.WriteLine "          </TABLE>"

End Sub

```

```

Sub ProcessBomFeature(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swBomFeat As SldWorks.BomFeature,
XMLfile As Scripting.TextStream)

    Dim swFeat As SldWorks.Feature
    Dim vTableArr As Variant
    Dim vTable As Variant
    Dim swTable As SldWorks.TableAnnotation

    Set swFeat = swBomFeat.GetFeature

    XMLfile.WriteLine "        <BOM>"
    XMLfile.WriteLine "            <NAME>" & swFeat.Name & "</NAME>"

    vTableArr = swBomFeat.GetTableAnnotations
    For Each vTable In vTableArr
        Set swTable = vTable
        ProcessTableAnn swApp, swModel, swTable, XMLfile
    Next vTable

    XMLfile.WriteLine "        </BOM>"

End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swDraw As SldWorks.DrawingDoc
    Dim swFeat As SldWorks.Feature
    Dim swBomFeat As SldWorks.BomFeature
    Dim sPathName As String
    Dim bIsFirstSheet As Boolean
    Dim fso As Scripting.FileSystemObject
    Dim XMLfile As Scripting.TextStream

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel

    bIsFirstSheet = True

    ' Strip off SOLIDWORKS file extension (slddrw)
    ' and add XML extension (xml)
    sPathName = swModel.GetPathName
    sPathName = Left(sPathName, Len(sPathName) - 6)
    sPathName = sPathName + ".xml"
    Set fso = CreateObject("Scripting.FileSystemObject")
    Set XMLfile = fso.CreateTextFile(sPathName, True)

    XMLfile.WriteLine "<BOMS>"

    Set swFeat = swModel.FirstFeature
    Do While Not swFeat Is Nothing
        If "DrSheet" = swFeat.GetType Then
            XMLfile.WriteLine "    <SHEET>"
            XMLfile.WriteLine "        <NAME>" & swFeat.Name & "</NAME>"

            bIsFirstSheet = False
        End If

        If "BomFeat" = swFeat.GetType Then
            Set swBomFeat = swFeat.GetSpecificFeature2
            ProcessBomFeature swApp, swModel, swBomFeat, XMLfile
        End If

        Set swFeat = swFeat.GetNextFeature
        If Not swFeat Is Nothing Then
            If "DrSheet" = swFeat.GetType And Not bIsFirstSheet Then
                XMLfile.WriteLine "    </SHEET>"
            End If
        End If
    Loop

    XMLfile.WriteLine "    </SHEET>"
    XMLfile.WriteLine "</BOMS>"
    XMLfile.Close

End Sub

```

Export BOMs to XML Example (VB.NET)

SOLIDWORKS Design manages Bills of Materials (BOM) and controls the information within them. You can extract this information for use in downstream systems such as ERP or other business systems.

In SOLIDWORKS 2004 and later, BOMs are now features and appear during a traversal of the FeatureManager design tree. This example shows how to get to each BOM in a drawing document and save the BOM information to an XML file. You can transform this XML file using XSL or transfer the file to other systems.

```
'-----
' Preconditions:
' 1. Open a drawing document with at least on BOM.
' 2. Add a reference to Microsoft Scripting Runtime (right-click
'    the name of the project in the Project Explorer and click Add Reference >
'    the Browse tab > C:\windows\system32\scrrun.dll > OK.
'
' Postconditions:
' 1. Saves an XML file to the folder where the drawing document resides,
'    overwriting any existing file of the same name.
' 2. Examine the folder where the drawing document resides.
'
' NOTES:
' * XML tags are based on BOM column headings.
' * Invalid characters must be removed from the
'   column headings.
' * XML schema is:
'   <BOMS>
'     <SHEET>
'       <NAME>Sheet1</NAME>
'     <BOM>
'       <NAME>Bill Of Materials1</NAME>
'       <TABLE>
'         <ROW>
'           <ITEM_NO>1</ITEM_NO>
'           <PART_NUMBER>2004_Part</PART_NUMBER>
'           <DESCRIPTION></DESCRIPTION>
'           <QTY>2</QTY>
'         </ROW>
'         <ROW>
'           <ITEM_NO>2</ITEM_NO>
'           <PART_NUMBER>2004_Part</PART_NUMBER>
'           <DESCRIPTION></DESCRIPTION>
'           <QTY>1</QTY>
'         </ROW>
'         <ROW>
'           <ITEM_NO>3</ITEM_NO>
'           <PART_NUMBER>2004_Part</PART_NUMBER>
'           <DESCRIPTION></DESCRIPTION>
'           <QTY>1</QTY>
'         </ROW>
'         <ROW>
'           <ITEM_NO>4</ITEM_NO>
'           <PART_NUMBER>bead7</PART_NUMBER>
'           <DESCRIPTION></DESCRIPTION>
'           <QTY>1</QTY>
'         </ROW>
'       </TABLE>
'     </BOM>
'   </SHEET>
'   <SHEET>
'     <NAME>Sheet2</NAME>
'     <BOM>
'       <NAME>Bill Of Materials2</NAME>
'       <TABLE>
'         <ROW>
'           <ITEM_NO>1</ITEM_NO>
'           <PART_NUMBER>Assem3</PART_NUMBER>
'           <DESCRIPTION></DESCRIPTION>
'           <QTY>1</QTY>
'         </ROW>
'         <ROW>
'           <ITEM_NO></ITEM_NO>
'           <PART_NUMBER>  cylinder</PART_NUMBER>
'           <DESCRIPTION></DESCRIPTION>
'           <QTY>1</QTY>
'         </ROW>
'         <ROW>
'           <ITEM_NO></ITEM_NO>
'           <PART_NUMBER>  cylinder</PART_NUMBER>
'           <DESCRIPTION></DESCRIPTION>
```

```

        <QTY>1</QTY>
    </ROW>
    <ROW>
        <ITEM_NO></ITEM_NO>
        <PART_NUMBER> SimpleCube_A</PART_NUMBER>
        <DESCRIPTION></DESCRIPTION>
        <QTY>1</QTY>
    </ROW>
    <ROW>
        <ITEM_NO></ITEM_NO>
        <PART_NUMBER> JoinedCyl</PART_NUMBER>
        <DESCRIPTION></DESCRIPTION>
        <QTY>1</QTY>
    </ROW>
</TABLE>
</BOM>
<BOM>
    <NAME>Bill Of Materials3</NAME>
    <TABLE>
        <ROW>
            <ITEM_NO>8</ITEM_NO>
            <PART_NUMBER>2004_Part</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>2</QTY>
        </ROW>
    </TABLE>
    <TABLE>
        <ROW>
            <ITEM_NO>9</ITEM_NO>
            <PART_NUMBER>2004_Part</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>1</QTY>
        </ROW>
        <ROW>
            <ITEM_NO>10</ITEM_NO>
            <PART_NUMBER>2004_Part</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>1</QTY>
        </ROW>
    </TABLE>
    <TABLE>
        <ROW>
            <ITEM_NO>11</ITEM_NO>
            <PART_NUMBER>bead7</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>1</QTY>
        </ROW>
    </TABLE>
</BOM>
<BOM>
    <NAME>Bill Of Materials4</NAME>
    <TABLE>
        <TITLE>BOM Table 2</TITLE>
        <ROW>
            <ITEM_NO>1</ITEM_NO>
            <PART_NUMBER>cylinder</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>1</QTY>
        </ROW>
        <ROW>
            <ITEM_NO>2</ITEM_NO>
            <PART_NUMBER>cylinder</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>1</QTY>
        </ROW>
    </TABLE>
    <TABLE>
        <TITLE>BOM Table 2</TITLE>
        <ROW>
            <ITEM_NO>3</ITEM_NO>
            <PART_NUMBER>SimpleCube_A</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>1</QTY>
        </ROW>
        <ROW>
            <ITEM_NO>4</ITEM_NO>
            <PART_NUMBER>JoinedCyl</PART_NUMBER>
            <DESCRIPTION></DESCRIPTION>
            <QTY>1</QTY>
        </ROW>
    </TABLE>
</BOM>
</SHEET>
</BOMS>

```

```
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
Imports Scripting
```

```
Partial Class SolidWorksMacro
```

```
Public Sub main()
```

```
Dim swModel As ModelDoc2
Dim swDraw As DrawingDoc
Dim swFeat As Feature
Dim swBomFeat As BomFeature
Dim sPathName As String
Dim bIsFirstSheet As Boolean
Dim fso As Scripting.FileSystemObject
Dim XMLfile As Scripting.TextStream

swModel = swApp.ActiveDoc
swDraw = swModel
bIsFirstSheet = True
' Strip off SOLIDWORKS file extension (slddrw)
' and add XML extension (xml)
sPathName = swModel.GetPathName
sPathName = Left(sPathName, Len(sPathName) - 6)
sPathName = sPathName + ".xml"
fso = CreateObject("Scripting.FileSystemObject")
XMLfile = fso.CreateTextFile(sPathName, True)
XMLfile.WriteLine("<BOMS>")
swFeat = swModel.FirstFeature
Do While Not swFeat Is Nothing
    If "DrSheet" = swFeat.GetTypeName Then
        XMLfile.WriteLine("    <SHEET>")
        XMLfile.WriteLine("        <NAME>" + swFeat.Name + "</NAME>")
        bIsFirstSheet = False
    End If
    If "BomFeat" = swFeat.GetTypeName Then
        swBomFeat = swFeat.GetSpecificFeature2
        ProcessBomFeature(swApp, swModel, swBomFeat, XMLfile)
    End If
    swFeat = swFeat.GetNextFeature
    If Not swFeat Is Nothing Then
        If "DrSheet" = swFeat.GetTypeName And Not bIsFirstSheet Then
            XMLfile.WriteLine("    </SHEET>")
        End If
    End If
Loop

XMLfile.WriteLine("    </SHEET>")
XMLfile.WriteLine("</BOMS>")
XMLfile.Close()
```

```
End Sub
```

```
Sub ProcessTableAnn(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swTableAnn As TableAnnotation, ByVal XMLfile As Scripting.TextStream)
```

```
Dim numRows As Integer
Dim numCol As Integer
Dim numHeader As Integer
Dim sHeaderText() As String
Dim j As Integer
Dim k As Integer
Dim nIndex As Integer
Dim nCount As Integer
Dim nStart As Integer
Dim nEnd As Integer
Dim nSplitDir As Integer
```

```
numHeader = swTableAnn.GetHeaderCount : Debug.Assert(numHeader >= 1)
nSplitDir = swTableAnn.GetSplitInformation(nIndex, nCount, nStart, nEnd)
If swTableSplitDirection_e.swTableSplit_None = nSplitDir Then
    Debug.Assert(0 = nIndex)
    Debug.Assert(0 = nCount)
    Debug.Assert(0 = nStart)
    Debug.Assert(0 = nEnd)
    numRows = swTableAnn.RowCount
    numCol = swTableAnn.ColumnCount
    nStart = numHeader
    nEnd = numRows - 1
Else
```

```
    Debug.Assert(swTableSplitDirection_e.swTableSplit_Horizontal = nSplitDir)
    Debug.Assert(nIndex >= 0)
    Debug.Assert(nCount >= 0)
    Debug.Assert(nStart >= 0)
```

```

        Debug.Assert(nEnd >= nStart)
        nNumCol = swTableAnn.ColumnCount
        If 1 = nIndex Then
            ' Add header offset for first portion of table
            nStart = nStart + nNumHeader
        End If
    End If
End If
XMLfile.WriteLine("                <TABLE>")
If swTableAnn.TitleVisible Then
    XMLfile.WriteLine("                <TITLE>" & swTableAnn.Title & "</TITLE>")
End If
ReDim sHeaderText(nNumCol - 1)
For j = 0 To nNumCol - 1
    sHeaderText(j) = swTableAnn.GetColumnTitle2(j, True)
    ' Replace invalid characters for XML tags
    sHeaderText(j) = Replace(sHeaderText(j), ".", "")
    sHeaderText(j) = Replace(sHeaderText(j), " ", "_")
Next j
For j = nStart To nEnd
    XMLfile.WriteLine("                <ROW>")
    For k = 0 To nNumCol - 1
        XMLfile.WriteLine("                " + "<" + sHeaderText(k) + ">" + swTableAnn.Text2(j, k, True) + "
</" + sHeaderText(k) + ">")
    Next k
    XMLfile.WriteLine("                </ROW>")
Next j
XMLfile.WriteLine("                </TABLE>")
End Sub
Sub ProcessBomFeature(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swBomFeat As BomFeature, ByVal XMLf
ile As Scripting.TextStream)
    Dim swFeat As Feature
    Dim vTableArr As Object
    Dim vTable As Object
    Dim swTable As TableAnnotation

    swFeat = swBomFeat.GetFeature
    XMLfile.WriteLine("                <BOM>")
    XMLfile.WriteLine("                <NAME>" & swFeat.Name & "</NAME>")
    vTableArr = swBomFeat.GetTableAnnotations
    For Each vTable In vTableArr
        swTable = vTable
        ProcessTableAnn(swApp, swModel, swTable, XMLfile)
    Next vTable
    XMLfile.WriteLine("                </BOM>")
End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Export Part to DWG Example (C#)

This example shows how to export sheet metal and annotation views of a part to DWG files.

```
//-----  
// Preconditions: Copy public_documents\samples\tutorial\api\2012-sm.sldprt to  
// c:\temp.  
//  
// Postconditions:  
// 1. Creates three DWG files containing:  
//   * Current annotation view  
//   * Front annotation view  
//   * flat-pattern sheet metal  
// 2. Locate and open the just-created DWG files in c:\temp.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
namespace ExportToDWG2_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
  
        ModelDoc2 swModel;  
        PartDoc swPart;  
        string sModelName;  
        string sPathName;  
        object varAlignment;  
        double[] dataAlignment = new double[12];  
        object varViews;  
        string[] dataViews = new string[2];  
  
        int options;  
  
        public void Main()  
        {  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
  
            sModelName = swModel.GetPathName();  
            sPathName = swModel.GetPathName();  
            sPathName = sPathName.Substring(0, sPathName.Length - 6);  
            sPathName = sPathName + "dwg";  
  
            swPart = (PartDoc)swModel;  
  
            dataAlignment[0] = 0.0;  
            dataAlignment[1] = 0.0;  
            dataAlignment[2] = 0.0;  
            dataAlignment[3] = 1.0;  
            dataAlignment[4] = 0.0;  
            dataAlignment[5] = 0.0;  
            dataAlignment[6] = 0.0;  
            dataAlignment[7] = 1.0;  
            dataAlignment[8] = 0.0;  
            dataAlignment[9] = 0.0;  
            dataAlignment[10] = 0.0;  
            dataAlignment[11] = 1.0;  
  
            varAlignment = dataAlignment;  
  
            dataViews[0] = "**Current";  
            dataViews[1] = "**Front";  
  
            varViews = dataViews;  
  
            //Export each annotation view to a separate drawing file
```

```
        swPart.ExportToDWG2(sPathName, sModelName,
(int)swExportToDWG_e.swExportToDWG_ExportAnnotationViews, false, varAlignment, false, false, 0,
varViews);

        //Export sheet metal to a single drawing file
        options = 1; //include flat-pattern geometry
        swPart.ExportToDWG2(sPathName, sModelName,
(int)swExportToDWG_e.swExportToDWG_ExportSheetMetal, true, varAlignment, false, false, options, null);
    }

    public SldWorks swApp;
}
}
```

Export Part to DWG Example (VBA)

This example shows how to export sheet metal and annotation views of a part to DWG files.

```
'-----  
' Preconditions: Copy public_documents\samples\tutorial\api\2012-sm.sldprt to  
' c:\temp.  
'  
' Postconditions:  
' 1. Creates three DWG files containing:  
'   * Current annotation view  
'   * Front annotation view  
'   * flat-pattern sheet metal  
' 2. Locate and open the just-created DWG files in c:\temp.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swPart As SldWorks.PartDoc  
Dim sModelName As String  
Dim sPathName As String  
Dim varAlignment As Variant  
Dim dataAlignment(11) As Double  
Dim varViews As Variant  
Dim dataViews(1) As String  
Dim options As Long  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    sModelName = swModel.GetPathName  
    sPathName = swModel.GetPathName  
    sPathName = Left(sPathName, Len(sPathName) - 6)  
    sPathName = sPathName + ".dwg"  
  
    Set swPart = swModel  
  
    dataAlignment(0) = 0#  
    dataAlignment(1) = 0#  
    dataAlignment(2) = 0#  
    dataAlignment(3) = 1#  
    dataAlignment(4) = 0#  
    dataAlignment(5) = 0#  
    dataAlignment(6) = 0#  
    dataAlignment(7) = 1#  
    dataAlignment(8) = 0#  
    dataAlignment(9) = 0#  
    dataAlignment(10) = 0#  
    dataAlignment(11) = 1#  
  
    varAlignment = dataAlignment  
  
    dataViews(0) = "*Current"  
    dataViews(1) = "*Front"  
  
    varViews = dataViews  
  
    'Export each annotation view to a separate drawing file  
    swPart.ExportToDWG2 sPathName, sModelName, swExportToDWG_ExportAnnotationViews, False,  
    varAlignment, False, False, 0, varViews
```

```
'Export sheet metal to a single drawing file
options = 1    'include flat-pattern geometry
swPart.ExportToDWG2 sPathName, sModelName, swExportToDWG_ExportSheetMetal, True, varAlignment,
False, False, options, Null
```

```
End Sub
```

Export Part to DWG Example (VB.NET)

This example shows how to export sheet metal and annotation views of a part to DWG files.

```
'-----  
' Preconditions: Copy public_documents\samples\tutorial\api\2012-sm.sldprt to  
' c:\temp.  
'  
' Postconditions:  
' 1. Creates three DWG files containing:  
'   * Current annotation view  
'   * Front annotation view  
'   * flat-pattern sheet metal  
' 2. Locate and open the just-created DWG files in c:\temp.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2
```

```
    Dim swPart As PartDoc
```

```
    Dim sModelName As String
```

```
    Dim sPathName As String
```

```
    Dim varAlignment As Object
```

```
    Dim dataAlignment(11) As Double
```

```
    Dim varViews As Object
```

```
    Dim dataViews(1) As String
```

```
    Dim options As Integer
```

```
    Sub main()
```

```
        swModel = swApp.ActiveDoc
```

```
        sModelName = swModel.GetPathName
```

```
        sPathName = swModel.GetPathName
```

```
        sPathName = Left(sPathName, Len(sPathName) - 6)
```

```
        sPathName = sPathName + "dwg"
```

```
        swPart = swModel
```

```
        dataAlignment(0) = 0.0#
```

```
        dataAlignment(1) = 0.0#
```

```
        dataAlignment(2) = 0.0#
```

```

        dataAlignment(3) = 1.0#
        dataAlignment(4) = 0.0#
        dataAlignment(5) = 0.0#
        dataAlignment(6) = 0.0#
        dataAlignment(7) = 1.0#
        dataAlignment(8) = 0.0#
        dataAlignment(9) = 0.0#
        dataAlignment(10) = 0.0#
        dataAlignment(11) = 1.0#

        varAlignment = dataAlignment

        dataViews(0) = "*Current"
        dataViews(1) = "*Front"

        varViews = dataViews

        'Export each annotation view to a
        separate drawing file
        swPart.ExportToDWG2(sPathName, sModelName, swExportToDWG_e.swExportToDWG_ExportAnnotationViews, False,
        varAlignment, False, False, 0, varViews)

        'Export sheet metal to a single
        drawing file
        options = 1    'include flat-pattern geometry
        swPart.ExportToDWG2(sPathName, sModelName, swExportToDWG_e.swExportToDWG_ExportSheetMetal, True,
        varAlignment, False, False,
        options, Nothing)

    End Sub

    Public swApp As SldWorks

End Class

```

Export SOLIDWORKS MBD to STEP 242 Example (C#)

This example shows how to export a SOLIDWORKS MBD part to a STEP 242 file.

```
//-----
// Preconditions:
// 1. Verify that:
//   * specified part,
//   * SOLIDWORKS MBD 3D PDF theme, and
//   * c:\temp exist.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part.
// 2. Gets the MBD3DPdfData object.
// 3. Sets the path and file name for the SOLIDWORKS MBD PDF.
// 4. Sets the theme for the SOLIDWORKS MBD 3D PDF.
// 5. Sets the standard views for the SOLIDWORKS MBD 3D PDF.
// 6. Publishes the part to SOLIDWORKS MBD PDF.
// 7. Exports the SOLIDWORKS MBD part document to STEP 242.
// 8. Examine the Immediate window and c:\temp.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            MBD3DPdfData swMBDPdfData = default(MBD3DPdfData);
            string fileName = null;
            object standardViews = null;
            int[] viewIDs = new int[3];
            int status = 0;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\block.sldprt";
            ;
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOp
            enDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Get MBD3DPdfData object
            swMBDPdfData = (MBD3DPdfData)swModelDocExt.GetMBD3DPdfData();

            //Set path and file name for SOLIDWORKS MBD 3D PDF
            swMBDPdfData.FilePath = "c:\\temp\\MyBlockMBD.PDF";

            //Set SOLIDWORKS MBD 3D PDF theme
            swMBDPdfData.ThemeName = "C:\\Program Files\\SolidWorks Corp\\SOLIDWORKS\\data\\themes\\simple part (a4, por
            trait)\\theme.xml";

            //Set standard views for SOLIDWORKS MBD 3D PDF
            viewIDs[0] = (int)swStandardViews_e.swFrontView;
            viewIDs[1] = (int)swStandardViews_e.swTopView;
            viewIDs[2] = (int)swStandardViews_e.swDimetricView;
            standardViews = (object)viewIDs;
            swMBDPdfData.SetStandardViews(standardViews);

            //Publish part document to SOLIDWORKS MBD 3D PDF
            status = swModelDocExt.PublishTo3DPDF(swMBDPdfData);
            Debug.Print("Status of publishing part to SOLIDWORKS MBD 3D PDF (0 = success): " + status);

            //Export SOLIDWORKS MBD part to STEP 242
            status = swModelDocExt.PublishSTEP242File("c:\\temp\\MyStepBlock.STP");
            Debug.Print("Status of exporting SOLIDWORKS MBD part to STEP 242 (0 = success): " + status);
        }
    }
}
```

```
    /// <summary>
    ///   The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Export SOLIDWORKS MBD to STEP 242 Example (VBA)

This example shows how to export a SOLIDWORKS MBD part to a STEP 242 file.

```
'-----  
' Preconditions:  
' 1. Verify that:  
'   * specified part,  
'   * SOLIDWORKS MBD 3D PDF theme, and  
'   * c:\temp exist.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified part.  
' 2. Gets the MBD3DPdfData object.  
' 3. Sets the path and file name for the SOLIDWORKS MBD PDF.  
' 4. Sets the theme for the SOLIDWORKS MBD 3D PDF.  
' 5. Sets the standard views for the SOLIDWORKS MBD 3D PDF.  
' 6. Publishes the part to SOLIDWORKS MBD PDF.  
' 7. Exports the SOLIDWORKS MBD part document to STEP 242.  
' 8. Examine the Immediate window and c:\temp.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swMBDPdfData As SldWorks.MBD3DPdfData  
Dim fileName As String  
Dim standardViews As Variant  
Dim viewIDs(2) As Long  
Dim status As Long  
Dim errors As Long  
Dim warnings As Long  
  
Sub main()  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block.sldprt"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "",  
errors, warnings)  
    Set swModelDocExt = swModel.Extension  
  
    'Get MBD3DPdfData object  
    Set swMBDPdfData = swModelDocExt.GetMBD3DPdfData  
  
    'Set path and file name for SOLIDWORKS MBD 3D PDF  
    swMBDPdfData.FilePath = "c:\temp\MyBlockMBD.PDF"  
  
    'Set SOLIDWORKS MBD 3D PDF theme  
    swMBDPdfData.ThemeName = "c:\Program Files\SolidWorks Corp\SOLIDWORKS\data\themes\simple part (a4,  
portrait)\theme.xml"  
  
    'Set standard views for SOLIDWORKS MBD 3D PDF  
    viewIDs(0) = swStandardViews_e.swFrontView  
    viewIDs(1) = swStandardViews_e.swTopView  
    viewIDs(2) = swStandardViews_e.swDimetricView  
    standardViews = viewIDs  
    swMBDPdfData.SetStandardViews (standardViews)  
  
    'Publish part document to SOLIDWORKS MBD 3D PDF  
    status = swModelDocExt.PublishTo3DPDF(swMBDPdfData)  
    Debug.Print ("Status of publishing part to SOLIDWORKS MBD 3D PDF (0 = success): " & status)  
  
    'Export SOLIDWORKS MBD part to STEP 242  
    status = swModelDocExt.PublishSTEP242File("c:\temp\MyStepBlock.STP")  
    Debug.Print ("Status of exporting SOLIDWORKS MBD part to STEP 242 (0 = success): " & status)  
  
End Sub
```

Export SOLIDWORKS MBD to STEP 242 Example (VB.NET)

This example shows how to export a SOLIDWORKS MBD part to a STEP 242 file.

```

'-----
' Preconditions:
' 1. Verify that:
'   * specified part,
'   * SOLIDWORKS MBD 3D PDF theme, and
'   * c:\temp exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part.
' 2. Gets the MBD3DPdfData object.
' 3. Sets the path and file name for the SOLIDWORKS MBD PDF.
' 4. Sets the theme for the SOLIDWORKS MBD 3D PDF.
' 5. Sets the standard views for the SOLIDWORKS MBD 3D PDF.
' 6. Publishes the part to SOLIDWORKS MBD PDF.
' 7. Exports the SOLIDWORKS MBD part document to STEP 242.
' 8. Examine the Immediate window and c:\temp.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swMBDPdfData As MBD3DPdfData
        Dim fileName As String
        Dim standardViews As Object
        Dim viewIDs(2) As Integer
        Dim status As Integer
        Dim errors As Integer
        Dim warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "",
errors, warnings)
        swModelDocExt = swModel.Extension

        'Get MBD3DPdfData object
        swMBDPdfData = swModelDocExt.GetMBD3DPdfData

        'Set path and file name for SOLIDWORKS MBD 3D PDF
        swMBDPdfData.FilePath = "c:\temp\MyBlockMBD.PDF"

        'Set SOLIDWORKS MBD 3D PDF theme
        swMBDPdfData.ThemeName = "C:\Program Files\SolidWorks Corp\SOLIDWORKS\data\themes\simple part (a4, portrait)\the
me.xml"

        'Set standard views for SOLIDWORKS MBD 3D PDF
        viewIDs(0) = swStandardViews_e.swFrontView
        viewIDs(1) = swStandardViews_e.swTopView
        viewIDs(2) = swStandardViews_e.swDimetricView
        standardViews = viewIDs
        swMBDPdfData.SetStandardViews(standardViews)

        'Publish part document to SOLIDWORKS MBD 3D PDF
        status = swModelDocExt.PublishTo3DPDF(swMBDPdfData)
        Debug.Print ("Status of publishing part to SOLIDWORKS MBD 3D PDF (0 = success): " & status)

        'Export SOLIDWORKS MBD part to STEP 242
        status = swModelDocExt.PublishSTEP242File("c:\temp\MyStepBlock.STP")
        Debug.Print("Status of exporting SOLIDWORKS MBD part to STEP 242 (0 = success): " & status)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.

```

```
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```

Extend Arms of Center Mark Examples (VBA)

This example shows how to extend the arms (handles) of a center mark.

```
'-----  
'  
' Preconditions: Drawing document is open and  
'             has a Drawing View1 with a center mark.  
'  
' Postconditions: The bottom and right-side arms are extended.  
'  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swDraw As SldWorks.DrawingDoc  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swModelDocExt As SldWorks.ModelDocExtension  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swCenterMark As SldWorks.CenterMark  
    Dim dExt2 As Double, dExt3 As Double  
    Dim boolstatus As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swModelDocExt = swModel.Extension  
    Set swDraw = swModel  
  
    ' Active the drawing view and select the center mark  
    boolstatus = swDraw.ActivateView("Drawing View1")  
    boolstatus = swModelDocExt.SelectByID2("DetailItem353@Drawing View1", "CENTERMARKSYM", 0.09197936270506, 0.1129166587377, 0, False, 0, Nothing, 0)  
    Set swCenterMark = swSelMgr.GetSelectedObject6(1, -1)  
    swModel.ClearSelection2 True  
  
    ' Get the lengths of the bottom and right-side arms of the center mark  
    dExt2 = swCenterMark.GetExtendedLength(0, swCenterMarkHandle_Down)  
    dExt3 = swCenterMark.GetExtendedLength(0, swCenterMarkHandle_Right)  
    Debug.Print "Length of arms before extension: " & dExt2 & "; " & dExt3  
  
    ' Extend the bottom and right-side arms of the center mark  
    boolstatus = swCenterMark.SetExtendedLength(0, swCenterMarkHandle_Down, 0.03)  
    boolstatus = swCenterMark.SetExtendedLength(0, swCenterMarkHandle_Right, 0.02)  
  
    ' Get the revised lengths of the bottom and right-side arms of the center mark  
    dExt2 = swCenterMark.GetExtendedLength(0, swCenterMarkHandle_Down)  
    dExt3 = swCenterMark.GetExtendedLength(0, swCenterMarkHandle_Right)  
    Debug.Print "Length of arms after extension: " & dExt2 & "; " & dExt3
```


Extend Sketch Entity Example (C#)

This example shows how to extend a selected sketch entity (e.g., line, segment, or arc) to meet another sketch entity.

```
//-----  
// Preconditions: Open a new part document.  
//  
// Postconditions:  
// 1. Inserts a new sketch is inserted.  
// 2. Creates two non-parallel lines.  
// 3. Extends the first line to meet the second line.  
// 4. Examine the graphics area.  
//-----  
  
using Microsoft.VisualBasic;  
  
using System;  
  
using System.Collections;  
  
using System.Collections.Generic;  
  
using System.Data;  
  
using System.Diagnostics;  
  
using SolidWorks.Interop.sldworks;  
  
using SolidWorks.Interop.swconst;  
  
using System.Runtime.InteropServices;  
  
namespace ExtendEntity_CSharp.csproj  
{  
  
    partial class SolidWorksMacro  
    {  
  
        ModelDoc2 swModel;  
  
        ModelDocExtension swModelDocExt;  
  
        SketchManager swSketchMgr;  
  
        bool boolstatus;  
  
        public void Main()  
        {  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
  
            swModelDocExt = swModel.Extension;  
  
            swSketchMgr = swModel.SketchManager;  
  
            swSketchMgr.InsertSketch(false);  
  
            // Create two non-parallel  
lines  
  
            swSketchMgr.CreateLine(-0.5,  
0.88, 0.0, -0.21, -0.13, 0.0);  
  
            swSketchMgr.CreateLine(-0.75, -1.128, 0.0, 0.41, -1.128,  
0.0);  
}
```

```

swModel.ViewZoomtofit2();

// Set the selection mode to default
swModel.SetPickMode();

// Select the sketch line to extend
boolstatus = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0.0, 0.0, 0.0, false, 0,
null, 0);

// Extend the selected sketch line to meet the second line
boolstatus = swSketchMgr.SketchExtend(0.0, 0.0, 0.0);

swSketchMgr.InsertSketch(true);

}

public SldWorks swApp;

}

```

Extend Sketch Entity Example (VBA)

This example shows how to extend a selected sketch entity (e.g., line, segment, or arc) to meet another sketch entity.

```
'-----  
' Preconditions: Open a new part document.  
'  
' Postconditions:  
' 1. Inserts a new sketch is inserted.  
' 2. Creates two non-parallel lines.  
' 3. Extends the first line to meet the second line.  
' 4. Examine the graphics area.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSketchMgr As SldWorks.SketchManager  
Dim boolstatus As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
    Set swSketchMgr = swModel.SketchManager  
  
    swSketchMgr.InsertSketch False  
    ' Create two non-parallel lines  
    swSketchMgr.CreateLine -0.5, 0.88, 0#, -0.21, -0.13, 0#  
    swSketchMgr.CreateLine -0.75, -1.128, 0#, 0.41, -1.128, 0#  
  
    swModel.ViewZoomtofit2  
    ' Set the selection mode to default  
    swModel.SetPickMode  
    ' Select the sketch line to extend  
    boolstatus = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0#, 0#, 0#, False, 0, Nothing, 0)  
    ' Extend the selected sketch line to meet the second line  
    boolstatus = swSketchMgr.SketchExtend(0#, 0#, 0#)  
  
    swSketchMgr.InsertSketch True  
  
End Sub
```

Extend Sketch Entity Example (VB.NET)

This example shows how to extend a selected sketch entity (e.g., line, segment, or arc) to meet another sketch entity.

```
'-----  
' Preconditions: Open a new part document.  
'  
' Postconditions:  
' 1. Inserts a new sketch is inserted.  
' 2. Creates two non-parallel lines.  
' 3. Extends the first line to meet the second line.  
' 4. Examine the graphics area.  
'-----  
  
Imports SolidWorks.Interop.sldworks  
  
Imports SolidWorks.Interop.swconst  
  
Imports System.Runtime.InteropServices  
  
Imports System  
  
Partial Class SolidWorksMacro  
  
    Dim swModel As ModelDoc2  
  
    Dim swModelDocExt As ModelDocExtension  
  
    Dim swSketchMgr As SketchManager  
  
    Dim boolstatus As Boolean  
  
    Sub main()  
  
        swModel = swApp.ActiveDoc  
  
        swModelDocExt = swModel.Extension  
  
        swSketchMgr = swModel.SketchManager  
  
        swSketchMgr.InsertSketch(False)  
  
        ' Create two non-parallel lines  
        swSketchMgr.CreateLine(-0.5,  
0.88, 0.0#, -0.21, -0.13, 0.0#)  
        swSketchMgr.CreateLine(-0.75, -1.128, 0.0#, 0.41, -1.128,  
0.0#)  
        swModel.ViewZoomtofit2  
  
        ' Set the selection mode to default  
        swModel.SetPickMode()  
  
        ' Select the sketch line to extend  
        boolstatus = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT",  
0.0#, 0.0#, 0.0#, False,  
0, Nothing,  
0)  
  
        ' Extend the selected sketch line
```

to meet the second line

```
        boolstatus = swSketchMgr.SketchExtend(0.0#,  
0.0#, 0.0#)
```

```
        swSketchMgr.InsertSketch(True)
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Extend Surface Example (VBA)

This example shows how to extend edges of a the selected surface.

```
'-----  
' Preconditions:  
' 1. Open a part that contains a surface body.  
' 2. Select an edge that you want to extend  
'    on the surface body.  
'  
' Postconditions:  
' 1. Creates and displays a temporary body using  
'    the selected edge.  
' 2. Examine the graphics area.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swPart As SldWorks.PartDoc  
Dim swEdge As SldWorks.Edge  
Dim swBody As SldWorks.Body2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swVertex As SldWorks.Vertex  
Dim swFace As SldWorks.Face2  
Dim edgeArr(0) As SldWorks.Edge  
Dim edgeVar As Variant  
Dim bExtLin As Boolean  
Dim endCond As Long  
Dim newBody1 As SldWorks.Body2  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swPart = swModel  
    Set swSelMgr = swModel.SelectionManager  
  
    ' Select an edge on the surface that you want to extend  
    Set swEdge = swSelMgr.GetSelectedObject5(1)  
    Set swBody = swEdge.GetBody  
    Set edgeArr(0) = swEdge  
    edgeVar = edgeArr  
    bExtLin = True  
    endCond = 0  
    Set swVertex = Nothing  
    Set swFace = Nothing  
    Set newBody1 = swBody.ExtendSurface((edgeVar), bExtLin, endCond, 0.01, swVertex, swFace)  
    newBody1.Display2 swPart, RGB(255, 0, 0), 0  
  
End Sub
```

Extract Configuration-specific Parameters Example (VBA)

This example shows how to extract and list all configuration-specific parameters from a model.

```
'-----  
' Preconditions:  
' 1. Open a part or assembly.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets configuration properties for each of the configurations  
'    in the part or assembly.  
' 2. Examine the Immediate window.  
'  
' NOTE: All parameters and their values are returned as strings.  
' It is up to the application to interpret the meaning of  
' each parameter and perform the appropriate conversion.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swConfigMgr As SldWorks.ConfigurationManager  
    Dim vConfName As Variant  
    Dim vConfParam As Variant  
    Dim vConfValue As Variant  
    Dim i As Long  
    Dim j As Long  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swConfigMgr = swModel.ConfigurationManager  
    Debug.Print "File = " + swModel.GetPathName  
  
    vConfName = swModel.GetConfigurationNames  
    For i = 0 To UBound(vConfName)  
        bRet = swConfigMgr.GetConfigurationParams(vConfName(i), vConfParam, vConfValue)  
        Debug.Assert bRet  
        Debug.Print " Configuration = " & vConfName(i)  
        If Not IsEmpty(vConfParam) Then  
            For j = 0 To UBound(vConfParam)  
                Debug.Print "      " & vConfParam(j) & " = " & vConfValue(j)  
            Next j  
        End If  
    Next i  
  
End Sub
```

Suppress or Unsuppress Feature Example (VBA)

This example shows how to suppress a feature in a part.

This example also shows how to perform a string comparison using Visual Basic. The first input parameter, searchStr, allows you to pass in a feature name or a portion of a feature name.

This macro cycles through all the features in the part and selects all features that contain the specified search string and suppresses those features.

```
' -----
' Preconditions: Verify that the part to open exists.
'
' Postconditions:
' 1. Opens the part.
' 2. Suppresses the specified feature.
' 3. Examine the FeatureManager design tree and
'    graphics area.
'
' NOTE: Because the part is used elsewhere, do not save
' changes.
' -----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swPart As SldWorks.PartDoc
    Dim swFeature As SldWorks.Feature
    Dim featureName As String
    Dim action As String
    Dim fileName As String
    Dim searchStr As String
    Dim errors As Long
    Dim warnings As Long
    Dim msg As String
    Dim style As Long
    Dim title As String
    Dim status As Boolean

    Set swApp = CreateObject("SldWorks.Application")

    ' Attach to the active document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\bolt.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "",
errors, warnings)
    searchStr = "chamf"
    action = "Suppress" 'Change to "Unsuppress" to unsuppress suppressed features

    Set swModelDocExt = swModel.Extension
    Set swPart = swModel

    ' Exit if no model is active
    If swModel Is Nothing Then
        Exit Sub
    End If

    ' Do not allow drawings or assemblies
    If (swModel.GetType <> swDocPART) Then
        ' Define message
        msg = "Macro Only valid for parts"
        ' OK button only
        style = VbMsgBoxStyle.vbOKOnly
        ' Define title
        title = "Error"
        ' Display error message
        Call MsgBox(msg, style, title)
        ' Exit this program
        Exit Sub
    End If

    ' Get the first feature in part
    Set swFeature = swPart.FirstFeature
```

```

' While a valid feature
Do While Not swFeature Is Nothing
    ' Get the name of the feature
    Let featureName = swFeature.Name
    ' See if the feature name contains search string
    If InStr(1, featureName, searchStr, 1) Then
        ' Select the feature
        status = swModelDocExt.SelectByID2(featureName, "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        If (action = "Suppress") Then
            status = swModel.EditSuppress2() ' Suppress the feature
        ElseIf (action = "Unsuppress") Then '
            status = swModel.EditUnsuppress2() ' Unsuppress the feature
        End If
    End If
    Set swFeature = swFeature.GetNextFeature() ' Get the next feature
Loop ' Continue looping until no more features

End Sub

```

FeatureTraversal.cpp Written in C++

```
// FeatureTraversal.cpp : Defines the entry point for the console application
// Traverses the FeatureManager design tree

#include "stdafx.h"

// Add the path to the SOLIDWORKS type libraries to the "Additional Include Directories".
#import "sldworks.tlb" raw_interfaces_only, raw_native_types, no_namespace, named_guids // SOLIDWORKS type library
#import "swconst.tlb" raw_interfaces_only, raw_native_types, no_namespace, named_guids // SOLIDWORKS constants type library

int _tmain(int argc, _TCHAR* argv[])
{
    // Initialize COM
    // Do this before using ATL smart pointers so COM is available.
    CoInitialize(NULL);

    // Use a block, so the smart pointers are destructed when the scope of this block is left
    {
        // Use ATL smart pointers
        CComPtr<ISldWorks> swApp;

        if(swApp.CoCreateInstance(__uuidof(SldWorks), NULL, CLSCTX_LOCAL_SERVER) != S_OK) {
            return(0);
        }

        swApp->put_UserControl(VARIANT_TRUE);
        swApp->put_Visible(VARIANT_TRUE);

        CComPtr<IModelDoc2> swModel;

        swApp->get_IActiveDoc2(&swModel);

        if (! swModel) {
            return(0);
        }

        CComBSTR strModelTitle;
        long nDocumentType; // swDocumentTypes_e

        swModel->GetTitle(&strModelTitle);
        swModel->GetType(&nDocumentType);

        CComPtr<IFeature> swFeature;
        CComPtr<IFeature> swSubFeature;
        CComBSTR strFeatureName;
        CComBSTR strFeatureType;

        swModel->IFirstFeature(&swFeature);

        while (swFeature) {
```

```

        swFeature->get_Name(&strFeatureName);
        swFeature->GetTypeNames2(&strFeatureType);

        swFeature->IGetFirstSubFeature(&swSubFeature);

        while (swSubFeature) {
            swSubFeature->get_Name(&strFeatureName);
            swSubFeature->GetTypeNames2(&strFeatureType);

            CComPtr<IFeature> swNextSubFeature;

            swSubFeature->IGetNextSubFeature(&swNextSubFeature);

            swSubFeature = swNextSubFeature;
        }

        CComPtr<IFeature> swNextFeature;

        swFeature->IGetNextFeature(&swNextFeature);

        swFeature = swNextFeature;
    }

}

// ATL smart pointers are destructed so all COM objects you held on to are released
// Now you can safely shutdown COM as you do not need it any longer

// Stop COM
CoUninitialize();

return(0);
}

```

FeatureTraversal.cpp Written in C++/CLI

```
// FeatureTraversal.cpp : main project file
// Traverses the FeatureManager design tree in the active model document

#include "stdafx.h"

using namespace System;

// Add the SOLIDWORKS primary interop assemblies to the reference
using namespace SOLIDWORKS::Interop::sldworks;
using namespace SOLIDWORKS::Interop::swconst;

int main(array<System::String ^> ^args)
{
    ISldWorks^ swApp;

    swApp = gcnew SldWorksClass;

    if (! swApp) {
        return(0);
    }

    swApp->UserControl = true;
    swApp->Visible = true;

    IModelDoc2^ swModel;

    swModel = swApp->IActiveDoc2;

    if (! swModel) {
        return(0);
    }

    String^ strModelTitle;
    swDocumentTypes_e nDocumentType;

    strModelTitle = swModel->GetTitle();
    nDocumentType = (swDocumentTypes_e)swModel->GetType();

    IFeature^ swFeature;
    IFeature^ swSubFeature;
    String^ strFeatureName;
    String^ strFeatureType;

    swFeature = swModel->IFirstFeature();

    while (swFeature) {

        strFeatureName = swFeature->Name;
        strFeatureType = swFeature->GetTypeName2();
```

```
swSubFeature = swFeature->IGetFirstSubFeature();

while (swSubFeature) {

    strFeatureName = swSubFeature->Name;
    strFeatureType = swSubFeature->GetType2();

    swSubFeature = swSubFeature->IGetNextSubFeature();
}

swFeature = swFeature->IGetNextFeature();
}

return(0);
}
```



```

        swCurve[i] = (Curve)swEdge.GetCurve();
    }
    vCurveArr = (Curve[])swCurve;
    swSurf = (Surface)swFace.GetSurface();
    swSurfCopy = (Surface)swSurf.Copy();
    swTempBody = (Body2)swSurfCopy.CreateTrimmedSheet4((vCurveArr), false);

    // Typically nothing is returned if the loop is
    // perpendicular to the surface, as in the
    // end loops of a cylinder

    if ((swTempBody != null))
    {
        // Sheet body only has one face
        swFeat = (Feature)swNewPart.CreateFeatureFromBody3(swTempBody, false, (int)swCreateFeatu
reBodyOpts_e.swCreateFeatureBodyCheck);
        swFeats[swFeats.GetUpperBound(0)] = swFeat;
        Debug.Assert((swFeats[swFeats.GetUpperBound(0)] != null));
        Array.Resize(ref swFeats, (swFeats.GetUpperBound(0)) + 2);
    }
    }
    }
    swLoop = (Loop2)swLoop.GetNext();
}
swFace = (Face2)swFace.GetNextFace();
}

Array.Resize(ref swFeats, (swFeats.GetUpperBound(0)));

swNewModel.ClearSelection2(true);
for (i = 0; i <= (swFeats.GetUpperBound(0)); i++)
{
    bRet = swFeats[i].Select2(true, 1);
}

swNewFeatMgr.InsertSewRefSurface(true, false, false, 3.001639406912E-05, 0.0001);

// Make sure surfaces are successfully sewn together
swKnitFeat = (Feature)swNewModel.FeatureByPositionReverse(0);
bRet = swKnitFeat.Select2(false, 1);
swThickFeat = (Feature)swNewFeatMgr.FeatureBossThicken(0.00254, 0, 7471215, false, true, true, true);

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Fill Holes in Part Example (VBA)

In CAM drilling operations, it might be useful to deduce the appearance of an item before machining begins. This is slightly different than calculating the minimum amount of raw material required, i.e., the stock size. This example shows how to use some of the geometry- and topology-related methods and properties to fill all holes in a part.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\api\cover_datum.sldrpt.  
'  
' Postconditions:  
' 1. Creates a new part.  
' 2. Fills the holes in the new part.  
' 3. Click the surface-imported and thicken features, which were created  
'    by filling the holes, in the FeatureManager design tree and examine  
'    the part after each click.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swPart As SldWorks.PartDoc  
Dim swBody As SldWorks.Body2  
Dim swFace As SldWorks.Face2  
Dim swLoop As SldWorks.Loop2  
Dim vEdgeArr As Variant  
Dim swCurve() As SldWorks.Curve  
Dim vCurveArr As Variant  
Dim swEdge As SldWorks.Edge  
Dim swTempBody As SldWorks.Body2  
Dim swSurf As SldWorks.Surface  
Dim swSurfCopy As SldWorks.Surface  
Dim sPartTemplateName As String  
Dim swNewModel As SldWorks.ModelDoc2  
Dim swNewPart As SldWorks.PartDoc  
Dim swFeat() As SldWorks.Feature  
Dim swKnitFeat As SldWorks.Feature  
Dim swThickFeat As SldWorks.Feature  
Dim swNewFeatMgr As SldWorks.FeatureManager  
Dim i As Long  
Dim bRet As Boolean  
Dim vBodies As Variant  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swPart = swModel  
  
vBodies = swPart.GetBodies2(swSolidBody, False)  
Set swBody = vBodies(0)  
  
' Create new part  
sPartTemplateName = swApp.GetUserPreferenceStringValue(swDefaultTemplatePart)  
Set swNewModel = swApp.NewDocument(sPartTemplateName, swDwgPaperASize, 0#, 0#)  
Set swNewFeatMgr = swNewModel.FeatureManager  
Set swNewPart = swNewModel  
  
ReDim swFeat(0)  
  
Set swFace = swBody.GetFirstFace  
Do While Not swFace Is Nothing  
    Set swLoop = swFace.GetFirstLoop  
    Do While Not swLoop Is Nothing  
        If swLoop.IsOuter Then  
            vEdgeArr = swLoop.GetEdges  
            If UBound(vEdgeArr) >= 0 Then  
                ReDim swCurve(UBound(vEdgeArr))  
                For i = 0 To UBound(vEdgeArr)  
                    Set swEdge = vEdgeArr(i)  
                    Set swCurve(i) = swEdge.GetCurve  
                Next i  
                vCurveArr = swCurve  
                Set swSurf = swFace.GetSurface
```

```

Set swSurfCopy = swSurf.Copy
Set swTempBody = swSurfCopy.CreateTrimmedSheet4(vCurveArr, False)
' Typically nothing is returned if the loop is
' perpendicular to the surface, as in the
' end loops of a cylinder
If Not swTempBody Is Nothing Then
    ' Sheet body only has one face
    Debug.Assert 1 = swTempBody.GetFaceCount
    Debug.Assert swSheetBody = swTempBody.GetType
    Set swFeat(UBound(swFeat)) = swNewPart.CreateFeatureFromBody3(swTempBody, False,
swCreateFeatureBodyCheck)
    Debug.Assert Not swFeat(UBound(swFeat)) Is Nothing
    ReDim Preserve swFeat(UBound(swFeat) + 1)
End If
End If
Set swLoop = swLoop.GetNext
Loop
Set swFace = swFace.GetNextFace
Loop
' Remove last empty feature
ReDim Preserve swFeat(UBound(swFeat) - 1)
swNewModel.ClearSelection2 True
For i = 0 To UBound(swFeat)
    bRet = swFeat(i).Select2(True, 1)
Next i
swNewFeatMgr.InsertSewRefSurface True, False, False, 3.001639406912E-05, 0.0001
' Make sure surfaces are successfully sewn together
Set swKnitFeat = swNewModel.FeatureByPositionReverse(0)
bRet = swKnitFeat.Select2(False, 1)
Set swThickFeat = swNewFeatMgr.FeatureBossThicken(0.00254, 0, 7471215, False, True, True, True)

End Sub

```

Fill Holes in Part Example (VB.NET)

In CAM drilling operations, it might be useful to deduce the appearance of an item before machining begins. This is slightly different than calculating the minimum amount of raw material required, i.e., the stock size. This example shows how to use some of the geometry- and topology-related methods and properties to fill all holes in a part.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\api\cover_datum.sldrpt.
'
' Postconditions:
' 1. Creates a new part.
' 2. Fills the holes in the new part.
' 3. Click the surface-imported and thicken features, which were created
'    by filling the holes, in the FeatureManager design tree and examine
'    the part after each click.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System.Diagnostics
Imports System

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swPart As PartDoc
        Dim swBody As Body2
        Dim swFace As Face2
        Dim swLoop As Loop2
        Dim vEdgeArr As Object
        Dim swCurve() As Curve
        Dim vCurveArr As Object
        Dim swEdge As Edge
        Dim swTempBody As Body2
        Dim swSurf As Surface
        Dim swSurfCopy As Surface
        Dim sPartTemplateName As String
        Dim swNewModel As ModelDoc2
        Dim swNewPart As PartDoc
        Dim swFeat() As Feature
        Dim swKnitFeat As Feature
        Dim swThickFeat As Feature
        Dim swNewFeatMgr As FeatureManager
        Dim i As Integer
        Dim bRet As Boolean
        Dim vBodies As Object

        swModel = swApp.ActiveDoc
        swPart = swModel

        vBodies = swPart.GetBodies2(swBodyType_e.swSolidBody, False)
        swBody = vBodies(0)

        ' Create new part
        sPartTemplateName = swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swDefaultTemplatePart)
        swNewModel = swApp.NewDocument(sPartTemplateName, swDwgPaperSizes_e.swDwgPaperASize, 0.0#, 0.0#)
        swNewFeatMgr = swNewModel.FeatureManager
        swNewPart = swNewModel

        ReDim swFeat(0)

        swFace = swBody.GetFirstFace
        Do While Not swFace Is Nothing
            swLoop = swFace.GetFirstLoop
            Do While Not swLoop Is Nothing
                If swLoop.IsOuter Then
                    vEdgeArr = swLoop.GetEdges
                    If UBound(vEdgeArr) >= 0 Then
                        ReDim swCurve(UBound(vEdgeArr))
                        For i = 0 To UBound(vEdgeArr)
                            swEdge = vEdgeArr(i)
                            swCurve(i) = swEdge.GetCurve
                        Next i
                        vCurveArr = swCurve
                        swSurf = swFace.GetSurface
                        swSurfCopy = swSurf.Copy
                        swTempBody = swSurfCopy.CreateTrimmedSheet4(vCurveArr, False)

                        ' Typically nothing is returned if the loop is
                        ' perpendicular to the surface, as in the
                        ' end loops of a cylinder
                    End If
                End If
                swLoop = swLoop.NextLoop
            Loop
            swFace = swFace.NextFace
        Loop
```

```

        If Not swTempBody Is Nothing Then
            ' Sheet body only has one face
            ' Debug.Assert(1 = swTempBody.GetFaceCount)
            ' Debug.Assert(swBodyType_e.swSheetBody = swTempBody.GetType)
            swFeat(UBound(swFeat)) = swNewPart.CreateFeatureFromBody3(swTempBody, False, swCreateFeature
BodyOpts_e.swCreateFeatureBodyCheck)
            ' Debug.Assert(Not swFeat(UBound(swFeat)) Is Nothing)
            ReDim Preserve swFeat(UBound(swFeat) + 1)
        End If
    End If
    End If
    swLoop = swLoop.GetNext
Loop
    swFace = swFace.GetNextFace
Loop

' Remove last empty feature
ReDim Preserve swFeat(UBound(swFeat) - 1)

swNewModel.ClearSelection2(True)
For i = 0 To UBound(swFeat)
    bRet = swFeat(i).Select2(True, 1)
Next i

swNewFeatMgr.InsertSewRefSurface(True, False, False, 0.00003001639406912, 0.0001)

' Make sure surfaces are successfully sewn together
swKnitFeat = swNewModel.FeatureByPositionReverse(0)
bRet = swKnitFeat.Select2(False, 1)
swThickFeat = swNewFeatMgr.FeatureBossThicken(0.00254, 0, 7471215, False, True, True, True)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Find Attribute Example (C#)

This example shows how to find an attribute on the selected entity.

```
//-----
// Preconditions:
// 1. Verify that the specified part document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Selects Cut-Extrude3.
// 3. Adds attribute att 1 to the selected feature.
// 4. Rebuilds the part.
// 5. Finds attribute att 1.
// 6. Examine the Immediate window to verify step 5.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace FindAttributeCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            //1 = invisible
            //0 = visible
            const int CreateVisible = 0;
            const string AttDefName = "XML_string";
            const string AttLenDimName = "att_name";
            const string AttLenValueName = "att_value";
            const string AttRootName = "att";
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = null;
            Entity swEnt = default(Entity);
            AttributeDef swAttDef = default(AttributeDef);
            SolidWorks.Interop.sldworks.Attribute swAtt = null;
            Parameter swParamName = default(Parameter);
            Parameter swParamValue = default(Parameter);
            string AttName = "";
            int i = 0;
            bool bRet = false;
            string fileName = null;
            int errors = 0;
            int warnings = 0;

            //Open part document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\smartcomponents\\bearing.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOp
enDocOptions_Silent, "", ref errors, ref warnings);

            //Select feature
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            bRet = swModelDocExt.SelectByID2("Cut-Extrude3", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swEnt = (Entity)swFeat;

            //Set attribute att 1
            swAttDef = (AttributeDef)swApp.DefineAttribute(AttDefName);
            bRet = swAttDef.AddParameter(AttLenDimName, (int)swParamType_e.swParamTypeString, 0.0, 0);
            bRet = swAttDef.AddParameter(AttLenValueName, (int)swParamType_e.swParamTypeDouble, 0.0, 0);
            bRet = swAttDef.Register();
            while (swAtt == null)
            {
                // Get a unique attribute name
                i = i + 1;
                AttName = AttRootName + " " + i.ToString();
                swAtt = (SolidWorks.Interop.sldworks.Attribute)swAttDef.CreateInstance5(swModel, swEnt, AttName, CreateV
isible, (int)swInConfigurationOpts_e.swThisConfiguration);
            }
            swParamName = (Parameter)swAtt.GetParameter(AttLenDimName);
            swParamValue = (Parameter)swAtt.GetParameter(AttLenValueName);
            bRet = swParamName.SetStringVal2(AttName + " -
" + AttLenDimName, (int)swInConfigurationOpts_e.swAllConfiguration, "");
        }
    }
}
```

```

bRet = swParamValue.SetDoubleValue2(i * 10, (int)swInConfigurationOpts_e.swAllConfiguration, "");
if ((swAtt != null))
{
    Debug.Print("Attribute " + AttName + " created.");
}
else
{
    Debug.Print("Attribute " + AttName + " not created.");
}

swModel.ForceRebuild3(true);

//Find attribute att 1
bRet = swModelDocExt.SelectByID2("att 1", "ATTRIBUTE", 0, 0, 0, false, 0, null, 0);
swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);

//Limit search in case attribute does not exist
while (swAtt == null & i < 300)
{
    swAtt = (SolidWorks.Interop.sldworks.Attribute)swEnt.FindAttribute(swAttDef, i);
    i = i + 1;
}
if ((swAtt != null))
{
    if (false == swAtt.GetEntityState((int)swAssociatedEntityStates_e.swIsEntityInvalid) & false == swAtt.Ge
tEntityState((int)swAssociatedEntityStates_e.swIsEntitySuppressed) & false == swAtt.GetEntityState((int)swAssociatedEnti
tyStates_e.swIsEntityAmbiguous) & false == swAtt.GetEntityState((int)swAssociatedEntityStates_e.swIsEntityDeleted))
    {
        swParamName = (Parameter)swAtt.GetParameter(AttLenDimName);
        swParamValue = (Parameter)swAtt.GetParameter(AttLenValueName);
        Debug.Print("Attribute " + swParamName.GetStringValue() + " found.");
        Debug.Print(" Value = " + swParamValue.GetDoubleValue());
    }
    else
    {
        Debug.Print(" Attribute found, but problems exist.");
    }
}
else
{
    Debug.Print(" Attribute not found.");
}
}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Find Attribute Example (VBA)

This example shows how to find an attribute on the selected entity.

```
'-----
' Preconditions:
' 1. Verify that the specified part document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Selects Cut-Extrude3.
' 3. Adds attribute att 1 to the selected feature.
' 4. Rebuilds the part.
' 5. Finds attribute att 1.
' 6. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim traverseLevel As Integer
Dim expandThis As Boolean

Sub main()

    '1 = invisible
    '0 = visible
    Const CreateVisible As Long = 0
    Const AttDefName As String = "XML_string"
    Const AttLenDimName As String = "att_name"
    Const AttLenValueName As String = "att_value"
    Const AttRootName As String = "att"
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swEnt As SldWorks.Entity
    Dim swAttDef As SldWorks.AttributeDef
    Dim swAtt As SldWorks.Attribute
    Dim swParamName As SldWorks.Parameter
    Dim swParamValue As SldWorks.Parameter
    Dim AttName As String
    Dim i As Long
    Dim bRet As Boolean
    Dim fileName As String
    Dim errors As Long
    Dim warnings As Long

    Set swApp = CreateObject("SldWorks.Application")

    'Open part document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\smartcomponents\bearing.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "",
    errors, warnings)

    'Select feature
    Set swModelDocExt = swModel.Extension
    bRet = swModelDocExt.SelectByID2("Cut-Extrude3", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swEnt = swFeat

    'Set attribute att 1
    Set swAttDef = swApp.DefineAttribute(AttDefName)
    bRet = swAttDef.AddParameter(AttLenDimName, swParamTypeString, 0#, 0)
    bRet = swAttDef.AddParameter(AttLenValueName, swParamTypeDouble, 0#, 0)
    bRet = swAttDef.Register
    While swAtt Is Nothing
        ' Get a unique attribute name
        i = i + 1
        AttName = AttRootName + Str(i)
        Set swAtt = swAttDef.CreateInstance5(swModel, swEnt, AttName, CreateVisible, swThisConfiguration)
```

```

Wend
Set swParamName = swAtt.GetParameter(AttLenDimName)
Set swParamValue = swAtt.GetParameter(AttLenValueName)
bRet = swParamName.SetStringValue2(AttName & " - " & AttLenDimName, swAllConfiguration, "")
bRet = swParamValue.SetDoubleValue2(i * 10, swAllConfiguration, "")
If Not swAtt Is Nothing Then
    Debug.Print "Attribute " & AttName & " created."
Else
    Debug.Print "Attribute " & AttName & " not created."
End If

swModel.ForceRebuild3 True

'Find attribute att 1
bRet = swModelDocExt.SelectByID2("att 1", "ATTRIBUTE", 0, 0, 0, False, 0, Nothing, 0)
Set swEnt = swSelMgr.GetSelectedObject6(1, -1)

'Limit search in case attribute does not exist
While swAtt Is Nothing And i < 300
    Set swAtt = swEnt.FindAttribute(swAttDef, i)
    i = i + 1
Wend

If Not swAtt Is Nothing Then
    If False = swAtt.GetEntityState(swIsEntityInvalid) And False = swAtt.GetEntityState(swIsEntitySuppressed) And
False = swAtt.GetEntityState(swIsEntityAmbiguous) And False = swAtt.GetEntityState(swIsEntityDeleted) Then
        Set swParamName = swAtt.GetParameter(AttLenDimName)
        Set swParamValue = swAtt.GetParameter(AttLenValueName)
        Debug.Print "Attribute " & swParamName.GetStringValue & " found."
        Debug.Print "    Value = " & swParamValue.GetDoubleValue
    Else
        Debug.Print "    Attribute found, but problems exist."
    End If
Else
    Debug.Print "    Attribute not found."
End If

End Sub

```

Find Attribute Example (VB.NET)

This example shows how to find an attribute on the selected entity.

```

'-----
' Preconditions:
' 1. Verify that the specified part document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Selects Cut-Extrude3.
' 3. Adds attribute att 1 to the selected feature.
' 4. Rebuilds the part.
' 5. Finds attribute att 1.
' 6. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        '1 = invisible
        '0 = visible
        Const CreateVisible As Integer = 0
        Const AttDefName As String = "XML_string"
        Const AttLenDimName As String = "att_name"
        Const AttLenValueName As String = "att_value"
        Const AttRootName As String = "att"
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swFeat As Feature
        Dim swEnt As Entity
        Dim swAttDef As AttributeDef
        Dim swAtt As SolidWorks.Interop.sldworks.Attribute = Nothing
        Dim swParamName As Parameter
        Dim swParamValue As Parameter
        Dim AttName As String = ""
        Dim i As Integer
        Dim bRet As Boolean
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer

        'Open part document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\smartcomponents\bearing.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "",
errors, warnings)

        'Select feature
        swModelDocExt = swModel.Extension
        bRet = swModelDocExt.SelectByID2("Cut-Extrude3", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swSelMgr = swModel.SelectionManager
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swEnt = swFeat

        'Set attribute att 1
        swAttDef = swApp.DefineAttribute(AttDefName)
        bRet = swAttDef.AddParameter(AttLenDimName, swParamType_e.swParamTypeString, 0.0#, 0)
        bRet = swAttDef.AddParameter(AttLenValueName, swParamType_e.swParamTypeDouble, 0.0#, 0)
        bRet = swAttDef.Register
        While swAtt Is Nothing
            ' Get a unique attribute name
            i = i + 1
            AttName = AttRootName + Str(i)
            swAtt = swAttDef.CreateInstance5(swModel, swEnt, AttName, CreateVisible, swInConfigurationOpts_e.swThisConfi
guration)
        End While
        swParamName = swAtt.GetParameter(AttLenDimName)
        swParamValue = swAtt.GetParameter(AttLenValueName)
        bRet = swParamName.SetStringValue2(AttName & " -
" & AttLenDimName, swInConfigurationOpts_e.swAllConfiguration, "")
        bRet = swParamValue.SetDoubleValue2(i * 10, swInConfigurationOpts_e.swAllConfiguration, "")
        If Not swAtt Is Nothing Then
            Debug.Print("Attribute " & AttName & " created.")
        End If
    End Sub
End Class

```

```

Else
    Debug.Print("Attribute " & AttName & " not created.")
End If

swModel.ForceRebuild3(True)

'Find attribute att 1
bRet = swModelDocExt.SelectByID2("att 1", "ATTRIBUTE", 0, 0, 0, False, 0, Nothing, 0)
swFeat = swSelMgr.GetSelectedObject6(1, -1)

'Limit search in case attribute does not exist
While swAtt Is Nothing And i < 300
    swAtt = swEnt.FindAttribute(swAttDef, i)
    i = i + 1
End While
If Not swAtt Is Nothing Then
    If False = swAtt.GetEntityState(swAssociatedEntityStates_e.swIsEntityInvalid) And False = swAtt.GetEntityState(swAssociatedEntityStates_e.swIsEntitySuppressed) And False = swAtt.GetEntityState(swAssociatedEntityStates_e.swIsEntityAmbiguous) And False = swAtt.GetEntityState(swAssociatedEntityStates_e.swIsEntityDeleted) Then
        swParamName = swAtt.GetParameter(AttLenDimName)
        swParamValue = swAtt.GetParameter(AttLenValueName)
        Debug.Print("Attribute " & swParamName.GetStringValue & " found.")
        Debug.Print("    Value = " & swParamValue.GetDoubleValue)
    Else
        Debug.Print("    Attribute found, but problems exist.")
    End If
Else
    Debug.Print("    Attribute not found.")
End If

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Find Minimum Radius Curvature of Surface (VBA)

This example shows how to find the minimum radius of curvature for the selected surface.

```
' -----  
' Preconditions: Model document is open and a surface is selected.  
'  
' Postconditions: None  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swDoc As SldWorks.ModelDoc2  
Dim selMgr As SldWorks.SelectionMgr  
Dim swface As SldWorks.Face2  
Dim swSurface As SldWorks.Surface  
Dim paraVar As Variant  
Dim uBoundArray(1) As Double  
Dim vBoundArray(1) As Double  
Dim test As Double  
Dim status As Boolean  
Dim uBoundSuf As Variant  
Dim vBoundSuf As Variant  
Dim vFaceUV As Variant  
Dim numOfRadius As Long  
Dim radius As Variant  
Dim location As Variant  
Dim uvparameter As Variant  
Dim locationDisp1 As SldWorks.MathPoint  
Dim arrayDataLoc1 As Variant  
Dim locationDisp2 As SldWorks.MathPoint  
Dim arrayDataLoc2 As Variant  
Dim uvParamDisp1 As SldWorks.MathPoint  
Dim arrayDataUV1 As Variant  
Dim uvParamDisp2 As SldWorks.MathPoint  
Dim arrayDataUV2 As Variant  
Dim arrayData1 As Variant  
Dim arrayData2 As Variant  
Dim I As Long  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swDoc = swApp.ActiveDoc  
Set selMgr = swDoc.SelectionManager  
Set swface = selMgr.GetSelectedObject6(1, -1)
```

```

Set swSurface = swface.GetSurface
vFaceUV = swface.GetUVBounds
paraVar = swSurface.Parameterization

uBoundArray(0) = vFaceUV(0)
uBoundArray(1) = vFaceUV(1)
vBoundArray(0) = vFaceUV(2)
vBoundArray(1) = vFaceUV(3)
uBoundSuf = uBoundArray
vBoundSuf = vBoundArray
status = swSurface.FindMinimumRadius(uBoundSuf, vBoundSuf, numOfRadius, radius, location, uvparameter)

'Assume that numOfRadius output is 2

Set locationDisp1 = location(0)
arrayData1 = locationDisp1.ArrayData
Set locationDisp2 = location(1)
arrayData2 = locationDisp2.ArrayData

Set uvParamDisp1 = uvparameter(0)
arrayData1 = uvParamDisp1.ArrayData
Set uvParamDisp2 = uvparameter(1)
arrayData2 = uvParamDisp2.ArrayData

Debug.Print ("Printing radius:-")
For I = 0 To (numOfRadius - 1)
    Debug.Print (radius(I))
Next I

Debug.Print ("Location")
Debug.Print (arrayData1(0))
Debug.Print (arrayData1(1))
Debug.Print (arrayData1(2))
Debug.Print (arrayData2(0))
Debug.Print (arrayData2(1))
Debug.Print (arrayData2(2))

Debug.Print ("UVParameter")
Debug.Print (arrayData1(0))
Debug.Print (arrayData1(1))
Debug.Print (arrayData1(2))
Debug.Print (arrayData2(0))
Debug.Print (arrayData2(1))
Debug.Print (arrayData2(2))

End Sub

```

Find Minimum Radius of Curve (VBA)

This example shows how to get the minimum radius of the selected curve.

```
' -----  
' Preconditions: Model document is open and a curve  
'               on a part is selected.  
'  
' Postconditions: None  
' -----  
  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swDoc As SldWorks.ModelDoc2  
Dim selMgr As SldWorks.SelectionMgr  
Dim swEdge As SldWorks.Edge  
Dim swCurve As SldWorks.Curve  
Dim param(1) As Double  
Dim isClose As Boolean  
Dim isPeriodic As Boolean  
Dim retPar As Boolean  
Dim test As Double  
Dim status As Boolean  
Dim BoundSuf As Variant  
Dim numOfRadius As Long  
Dim radius As Variant  
Dim location As Variant  
Dim uvparameter As Variant  
Dim locationDisp1 As SldWorks.MathPoint  
Dim arrayData1 As Variant  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swDoc = swApp.ActiveDoc  
Set selMgr = swDoc.SelectionManager  
Set swEdge = selMgr.GetSelectedObject6(1, -1)  
' Get the selected curve  
Set swCurve = swEdge.GetCurve  
  
retPar = swCurve.GetEndParams(param(0), param(1), isClose, isPeriodic)  
BoundSuf = param  
status = swCurve.FindMinimumRadius(BoundSuf, numOfRadius, radius, location, uvparameter)  
  
'Assume that numOfRadius output is 1
```

```
Debug.Print ("Radius")
Debug.Print (radius(0))

Set locationDisp1 = location(0)
arrayData1 = locationDisp1.ArrayData

Debug.Print ("Location")
Debug.Print (arrayData1(0))
Debug.Print (arrayData1(1))
Debug.Print (arrayData1(2))

Debug.Print ("Parameter")
Debug.Print (uvparameter(0))

End Sub
```

Find Outside Edges of Face Example (VBA)

This example shows how to find the outside edges of the selected face.

```
'-----  
' Preconditions:  
' 1. Open a part and select a face.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Finds the outside edges of the selected face  
'    and creates a 3D sketch using the outside edges.  
' 2. Examine the Immediate window, selected face,  
'    FeatureManager design tree, and 3D sketch.  
'-----
```

Option Explicit

```
Sub CreateTessCurve(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch, swTrimCurve  
As SldWorks.Curve)
```

```
    Const nChordTol As Double = 0.001    ' Meters  
    Const nLengthTol As Double = 0.001    ' Meters  
    Dim nStartParam As Double  
    Dim nEndParam As Double  
    Dim bIsClosed As Boolean  
    Dim bIsPeriodic As Boolean  
    Dim vStartPt As Variant  
    Dim vEndPt As Variant  
    Dim vTessPts As Variant  
    Dim swSketchSeg As SldWorks.SketchSegment  
    Dim bRet As Boolean  
    Dim i As Long
```

```
    ' Really not needed because curve is a trimmed curve,  
    ' so could pass in trim points as parameters  
    bRet = swTrimCurve.GetEndParams(nStartParam, nEndParam, bIsClosed, bIsPeriodic)  
    vStartPt = swTrimCurve.Evaluate(nStartParam)  
    vEndPt = swTrimCurve.Evaluate(nEndParam)
```

```
    vTessPts = swTrimCurve.GetTessPts(nChordTol, nLengthTol, (vStartPt), (vEndPt))
```

```
    ' Disable VBA range checking because tessellation points  
    ' might not be a multiple of 6
```

```
    On Error Resume Next
```

```
    For i = 0 To UBound(vTessPts) Step 3
```

```
        Set swSketchSeg = swModel.CreateLine2(vTessPts(i + 0), vTessPts(i + 1), vTessPts(i + 2), vTessPts(i + 3),  
vTessPts(i + 4), vTessPts(i + 5))  
    Next i
```

```
    On Error GoTo 0
```

```
End Sub
```

```
Sub CreateTessLoop(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swLoop As SldWorks.Loop2)
```

```
    Dim vEdgeArr As Variant  
    Dim vEdge As Variant  
    Dim swEdge As SldWorks.Edge  
    Dim swCurve As SldWorks.Curve  
    Dim swSketch As SldWorks.Sketch  
    Dim bRet As Boolean
```

```
    swModel.Insert3DSketch2 False
```

```
    swModel.SetAddToDB True
```

```
    swModel.SetDisplayWhenAdded False
```

```
    Set swSketch = swModel.GetActiveSketch2
```

```
    vEdgeArr = swLoop.GetEdges
```

```
    For Each vEdge In vEdgeArr
```

```
        Set swEdge = vEdge
```

```
        Set swCurve = swEdge.GetCurve
```

```
        CreateTessCurve swApp, swModel, swSketch, swCurve
```

```
    Next vEdge
```

```
    swModel.SetDisplayWhenAdded True
```

```
    swModel.SetAddToDB False
```

```
    swModel.Insert3DSketch2 True
```

```

        bRet = swModel.EditRebuild3

End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swPart As SldWorks.PartDoc
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFace As SldWorks.Face2
    Dim swLoop As SldWorks.Loop2
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swPart = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swFace = swSelMgr.GetSelectedObject6(1, -1)
    Debug.Print "FaceArea = " & swFace.GetArea * 1000000# & " mm^2"
    Debug.Print " LoopCount = " & swFace.GetLoopCount
    Debug.Print ""

    Set swLoop = swFace.GetFirstLoop
    Do While Not swLoop Is Nothing
        i = i + 1
        Debug.Print " Loop(" & i & ")"
        Debug.Print " IsOuter = " & swLoop.IsOuter
        Debug.Print " IsSingular = " & swLoop.IsSingular
        If swLoop.IsOuter Then
            CreateTessLoop swApp, swModel, swLoop
        End If
        Set swLoop = swLoop.GetNext
    Loop

End Sub

```

Find_Parent_Feature_using_Collections_Example_VB (Topic)

SOLIDWORKS API Help

Find Parent Feature using Collections Example (VBA)

This example shows how to walk the FeatureManager design tree and touch all of the parent features of a selected feature. Using collections lets you visit all of the parent features without using a recursive operation.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\weldment_box3.sldprt.  
' 2. Expand the Cut list(31) folder and click Gusset4.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Finds the parent features of the selected feature  
'    and selects that parent features in the graphics area.  
' 2. Examine the Immediate window and graphics area.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModelDoc2 As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swBody2 As SldWorks.Body2  
Dim swFace2 As SldWorks.Face2  
Dim swFeature As SldWorks.Feature  
Dim parents As Variant  
Dim varobj As Variant  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModelDoc2 = swApp.ActiveDoc  
    Set swSelMgr = swModelDoc2.SelectionManager  
    Set swBody2 = swSelMgr.GetSelectedObject6(1, -1)  
  
    Set swFace2 = swBody2.GetFirstFace  
    Set swFeature = swFace2.GetFeature  
    Debug.Print ("Selected feature name: " & swFeature.Name)  
    Debug.Print ("Selected feature type: " & swFeature.GetTypeName)  
    Debug.Print ("")  
    If "WeldMemberFeat" <> swFeature.GetTypeName Then  
        Dim parentcoll As New Collection  
        Set varobj = swFeature  
        parentcoll.Add swFeature  
        Do While parentcoll.Count > 0  
            Set swFeature = parentcoll(1)  
            If "WeldMemberFeat" <> swFeature.GetTypeName Then  
                parentcoll.Remove 1  
                parents = swFeature.GetParents  
                For Each varobj In parents  
                    Set swFeature = varobj  
                    Debug.Print (" Parent feature name: " & swFeature.Name)  
                    Debug.Print (" Parent feature type: " & swFeature.GetTypeName)  
                    Debug.Print ("")  
                    parentcoll.Add varobj  
                Next varobj  
                Set swFeature = Nothing  
            Else  
                Exit Do  
            End If  
        Loop  
    End If  
    If Not swFeature Is Nothing Then  
        swFeature.Select2 False, 0  
    Else  
        Stop  
    End If  
  
End Sub
```

Find_Total_Length_of_Sketch_Segments_in_Sketch_Example_VB (Topic)

SOLIDWORKS API Help

Find Total Length of Sketch Segments in Sketch Example (VBA)

This example shows how to find the total length of all sketch segments in a sketch.

```
'-----  
' Preconditions:  
' 1. Open a part and select a sketch.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the total length of the segments in the  
'    selected sketch.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Public Enum swSketchSegments_e  
    swSketchLINE = 0  
    swSketchARC = 1  
    swSketchELLIPSE = 2  
    swSketchSPLINE = 3  
    swSketchTEXT = 4  
    swSketchPARABOLA = 5  
End Enum  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swFeat As SldWorks.Feature  
    Dim swSketch As SldWorks.Sketch  
    Dim i As Long  
    Dim bRet As Boolean  
    Dim vSketchSeg As Variant  
    Dim swSketchSeg As SldWorks.SketchSegment  
    Dim nLength As Double  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeat = swSelMgr.GetSelectedObject5(1)  
    Set swSketch = swFeat.GetSpecificFeature2  
    vSketchSeg = swSketch.GetSketchSegments  
    For i = 0 To UBound(vSketchSeg)  
        Set swSketchSeg = vSketchSeg(i)  
        ' Ignore construction lines  
        If swSketchSeg.ConstructionGeometry = False Then  
            ' Ignore text  
            If swSketchTEXT <> swSketchSeg.GetType Then  
                nLength = nLength + swSketchSeg.GetLength  
            End If  
        End If  
    Next i  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print " Sketch = " & swFeat.Name  
    Debug.Print " Total length = " & nLength * 1000# & " mm"  
  
End Sub
```

Fire Selection After Selection Made Example (C#)

This example shows how to fire an event after a selection is made in a part, assembly, or drawing document.

```
//-----  
// Preconditions: Open a part, assembly, or drawing.  
//  
// Postconditions:  
// 1. Select an entity.  
// 2. Displays a message box confirming your  
//    selection.  
//    NOTE: If necessary, click the message box icon  
//          on the Windows taskbar to display the message box.  
// 3. Click OK to close the message box.  
//  
// NOTE: Tools > Options > System> Stop VSTA debugger  
// on macro exit must be cleared for this macro to run to  
// completion.  
//-----  
  
using SolidWorks.Interop.sldworks;  
  
using SolidWorks.Interop.swconst;  
  
using System;  
  
using System.Windows.Forms;  
  
namespace UserSelectionPostNotifyEventCSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public PartDoc pDoc;  
        public AssemblyDoc aDoc;  
        public DrawingDoc dDoc;  
  
        public void Main()  
        {  
            ModelDoc2 swModel;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
  
            // Determine the document type  
            // and set up  
event handlers  
            if (swModel.GetType() == (int)swDocumentTypes_e.swDocPART)  
            {  
                pDoc = (PartDoc)swModel;  
            }  
            else if (swModel.GetType() == (int)swDocumentTypes_e.swDocASSEMBLY)
```

```

    {
        aDoc = (AssemblyDoc)swModel;
    }
    else if (swModel.GetType() == (int)swDocumentTypes_e.swDocDRAWING)
    {
        dDoc = (DrawingDoc)swModel;
    }

    AttachEventHandlers();
}

public void AttachEventHandlers()
{
    AttachSWEvents();
}

public void AttachSWEvents()
{
    if ((pDoc != null))
    {
        pDoc.UserSelectionPostNotify += this.pDoc_UserSelectionPostNotify;
    }
    if ((aDoc != null))
    {
        aDoc.UserSelectionPostNotify += this.aDoc_UserSelectionPostNotify;
    }
    if ((dDoc != null))
    {
        dDoc.UserSelectionPostNotify += this.dDoc_UserSelectionPostNotify;
    }
}

private int pDoc_UserSelectionPostNotify()
{
    int functionReturnValue = 0;

    MessageBox.Show("An entity was selected  
in a part document.");

    return functionReturnValue;
}

```

```

public int aDoc_UserSelectionPostNotify()
{
    int functionReturnValue = 0;

    MessageBox.Show("An entity was selected
in a assembly document.");

    return functionReturnValue;
}

private int dDoc_UserSelectionPostNotify()
{
    int functionReturnValue = 0;

    MessageBox.Show("An entity was selected
in a drawing document.");

    return functionReturnValue;
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Fire Event After Selection Made Example (VBA)

This example shows how to fire an event after a selection is made in a part, assembly, or drawing document.

```
'-----  
' Preconditions:  
' 1. Copy Main into your macro.  
' 2. Click Insert > Class Module and copy Class1  
'    into that module.  
' 3. Click Insert > Class Module and copy Class2  
'    into that module.  
' 4. Click Insert > Class Module and copy Class3  
'    into that module.  
' 5. Open a part, assembly, or drawing.  
'  
' Postconditions:  
' 1. Select an entity.  
' 2. Displays a message box confirming your  
'    selection.  
' 3. Click OK to close the message box.  
'-----
```

[Main](#)

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim partDoc As New Class1  
Dim assemblyDoc As New Class2  
Dim drawingDoc As New Class3  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
  
' Determine the document type, then  
' execute its corresponding class module  
If swModel.GetType = swDocPART Then  
    partDoc.init swModel  
ElseIf swModel.GetType = swDocASSEMBLY Then  
    assemblyDoc.init swModel  
ElseIf swModel.GetType = swDocDRAWING Then  
    drawingDoc.init swModel  
End If  
  
End Sub
```

[Back to top](#)

[Class1](#)

```
Public WithEvents doc As partDoc  
  
Public Function init(ByRef docin As Object)  
    Set doc = docin  
End Function  
  
Private Function doc_UserSelectionPostNotify() As Long  
    MsgBox "Entity selected in a part document."  
End Function
```

[Back to top](#)

[Class2](#)

```
Public WithEvents doc As assemblyDoc
```

```
Public Function init(ByRef docin As Object)
    Set doc = docin
End Function

Private Function doc_UserSelectionPostNotify() As Long
    MsgBox "Entity selected in an assembly document."
End Function
```

[Back to top](#)

Class3

```
Public WithEvents doc As drawingDoc

Public Function init(ByRef docin As Object)
    Set doc = docin
End Function

Private Function doc_UserSelectionPostNotify() As Long
    MsgBox "Entity selected in a drawing document."
End Function
```

[Back to top](#)

Fire Event After Selection Made Example (VB.NET)

This example shows how to fire an event after a selection is made in a part, assembly, or drawing document.

```
'-----  
' Preconditions: Open a part, assembly, or drawing.  
'  
' Postconditions:  
' 1. Select an entity.  
' 2. Displays a message box confirming your  
'    selection.  
'    NOTE: If necessary, click the message box icon  
'          on the Windows taskbar to display the message box.  
' 3. Click OK to close the message box.  
'  
' NOTE: Tools > Options > System> Stop VSTA debugger  
'       on macro exit must be cleared for this macro to run to  
'       completion.  
'-----  
Imports SolidWorks.Interop.sldworks  
  
Imports SolidWorks.Interop.swconst  
  
Imports System  
  
Partial Class SolidWorksMacro  
  
    Public WithEvents pDoc As PartDoc  
  
    Public WithEvents aDoc As AssemblyDoc  
  
    Public WithEvents dDoc As DrawingDoc  
  
    Public Sub main()  
  
        Dim swModel As ModelDoc2  
  
        swModel = swApp.ActiveDoc  
  
        ' Determine the document type  
  
        ' and set up event  
handlers  
  
        If swModel.GetType = swDocumentTypes_e.swDocPART Then  
            pDoc = swModel  
  
        ElseIf swModel.GetType = swDocumentTypes_e.swDocASSEMBLY Then  
            aDoc = swModel  
  
        ElseIf swModel.GetType = swDocumentTypes_e.swDocDRAWING Then  
            dDoc = swModel  
  
        End If  
  
        AttachEventHandlers()
```

End Sub

```
Sub AttachEventHandlers()  
    AttachSWEvents()  
End Sub
```

```
Sub AttachSWEvents()  
    If Not pDoc Is Nothing Then  
        AddHandler pDoc.UserSelectionPostNotify, AddressOf Me.pDoc_UserSelectionPostNotify  
    End If  
  
    If Not aDoc Is Nothing Then  
        AddHandler aDoc.UserSelectionPostNotify, AddressOf Me.aDoc_UserSelectionPostNotify  
    End If  
  
    If Not dDoc Is Nothing Then  
        AddHandler dDoc.UserSelectionPostNotify, AddressOf Me.dDoc_UserSelectionPostNotify  
    End If  
End Sub
```

```
Private Function pDoc_UserSelectionPostNotify() As Integer  
    MsgBox("Entity selected in part document.")  
End Function
```

```
Public Function aDoc_UserSelectionPostNotify() As Integer  
    MsgBox("Entity selected in assembly document.")  
End Function
```

```
Private Function dDoc_UserSelectionPostNotify() As Integer  
    MsgBox("Entity selected in drawing document.")  
End Function
```

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

End Class

Fire Events for External Output Time Step Changes Example (C#)

This example shows how to listen and catch events for external force and motor output time step changes.

```
//-----
// Preconditions:
// 1. Start SOLIDWORKS Design Premium, which includes SOLIDWORKS Motion.
// 2. Verify that the c:\temp folder exists.
// 3. In SOLIDWORKS Design:
//    a. Start the SOLIDWORKS Motion Study add-in (in
//       SOLIDWORKS Design, click Tools > Add-Ins > SOLIDWORKS Motion).
//    b. Open public_documents\samples\tutorial\api\simple-block.sldasm.
//    c. Click MotionStudy 1 and select Motion Analysis in the
//       Type of Study list at the upper-left corner of the MotionManager.
// 4. In the IDE, reference the SOLIDWORKS Motion Study interop
//    assembly (right-click the name of the project in the Project
//    Explorer > click Add Reference > browse to
//    install_dir\api\redist > click SolidWorks.Interop.swmotionstudy).
//
// Postconditions:
// 1. Calculates motion analysis
// 2. Opens c:\temp\ForceAndMotorValuesCSharp.txt for output.
// 3. Sets forces and motors to external.
// 4. Recalculates motion analysis.
// 5. Fires an event for every force and motor output time step change
//    and recalculates and writes the force magnitude or motor speed to
//    c:\temp\ForceAndMotorValuesCSharp.txt.
// 6. Open and examine c:\temp\ForceAndMotorValuesCSharp.txt.
//
// NOTE: Because this model is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swmotionstudy;
using System.Runtime.InteropServices;

namespace MotionExternalForceMotorEventsCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public MotionStudy motionstudy;
        public MotionStudy swMotionStudy;
        public ModelDoc2 swModel;

        public void Main()
        {
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            MotionStudyManager swMotionMgr = default(MotionStudyManager);
            string studyName = null;
            int numStudies = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Motion study name
            studyName = "Motion Study 1";

            // Get the MotionManager
            swMotionMgr = (MotionStudyManager)swModelDocExt.GetMotionStudyManager();
            numStudies = swMotionMgr.GetMotionStudyCount();
            swMotionStudy = (MotionStudy)swMotionMgr.GetMotionStudy(studyName);
            if (!swMotionStudy.IsActive)
                swMotionStudy.Activate();

            //Calculate the motion study
```

```

swMotionStudy.Calculate();

// Set up events
motionstudy = (MotionStudy)swMotionStudy;
AttachEventHandlers();

// Set feature Force1 as external force
SetForceExternalFlag("Force1", true);

// Set feature LinearMotor1 as external motor
SetMotorExternalFlag("LinearMotor1", true);

// Fire event at every output time step
swMotionStudy.SetFireOutputTimeStepEvents(true);

//Recalculate the motion study
swMotionStudy.Calculate();
}

private void SetForceExternalFlag(string featureName, bool setFlag)
{
    // Set forces to external; i.e., listen for events

    Feature swFeature = default(Feature);
    SimulationForceFeatureData swSimulationFeatureData = default(SimulationForceFeatureData);
    string simTypeName = null;
    object[] motionFeatures = null;
    bool ret = false;
    int i = 0;

    motionFeatures = (object[])swMotionStudy.GetMotionFeatures();
    for (i = 0; i <= motionFeatures.GetUpperBound(0); i++)
    {
        swFeature = (Feature)motionFeatures[i];
        simTypeName = swFeature.GetTypeName();
        if (simTypeName == "AEMLinearForce" | simTypeName == "AEMTorque")
        {
            if (swFeature.Name == featureName)
            {
                System.IO.StreamWriter outputWrite = new
System.IO.StreamWriter("c:\\temp\\ForceAndMotorValuesCSharp.txt", true);

                swSimulationFeatureData = (SimulationForceFeatureData)swFeature.GetDefinition();
                swSimulationFeatureData.ExternalState = setFlag;
                ret = swFeature.ModifyDefinition(swSimulationFeatureData, swModel, null);
                outputWrite.WriteLine("Feature: " + swFeature.Name);
                outputWrite.WriteLine("  Listen for events: " +
swSimulationFeatureData.ExternalState);
                outputWrite.WriteLine(" ");

                outputWrite.Close();
            }
        }
    }
}

private void SetMotorExternalFlag(string featureName, bool setFlag)
{
    // Set motors to external; i.e., listen for events

    Feature swFeature = default(Feature);
    SimulationMotorFeatureData swSimulationFeatureData = default(SimulationMotorFeatureData);
    string simTypeName = null;
    object[] motionFeatures = null;
    int i = 0;
    bool ret = false;

    motionFeatures = (object[])swMotionStudy.GetMotionFeatures();
    for (i = 0; i <= motionFeatures.GetUpperBound(0); i++)
    {
        swFeature = (Feature)motionFeatures[i];
        simTypeName = swFeature.GetTypeName();
    }
}

```

```

        if (simTypeName == "AEMLinearMotor" | simTypeName == "AEMRotationalMotor")
        {
            if (swFeature.Name == featureName)
            {
                System.IO.StreamWriter outputWrite = new
System.IO.StreamWriter("c:\\temp\\ForceAndMotorValuesCSharp.txt", true);

                swSimulationFeatureData = (SimulationMotorFeatureData)swFeature.GetDefinition();
                swSimulationFeatureData.ExternalState = setFlag;
                ret = swFeature.ModifyDefinition(swSimulationFeatureData, swModel, null);
                outputWrite.WriteLine("Feature name: " + swFeature.Name);
                outputWrite.WriteLine(" Listen for events: " +
swSimulationFeatureData.ExternalState);
                outputWrite.WriteLine(" ");

                outputWrite.Close();
            }
        }
    }

    public void writeOut(object name, double value)
    // Write force and motor values to file
    {
        string Name;

        Name = (string)name;
        System.IO.StreamWriter outputWrite = new
System.IO.StreamWriter("c:\\temp\\ForceAndMotorValuesCSharp.txt", true);

        outputWrite.WriteLine("Name: " + Name);
        outputWrite.WriteLine(" Value: " + value.ToString());

        outputWrite.Close();
    }

    public void AttachEventHandlers()
    {
        AttachSWEvents();
    }

    public void AttachSWEvents()
    {
        motionstudy.ForceOutputTimeStepChangeNotify +=
this.motionStudy_ForceOutputTimeStepChangeNotify;
        motionstudy.MotorOutputTimeStepChangeNotify +=
this.motionStudy_MotorOutputTimeStepChangeNotify;
        motionstudy.OutputTimeStepChangeNotify += this.motionStudy_OutputTimeStepChangeNotify;
        motionstudy.StartCalculateNotify += this.motionStudy_StartCalculateNotify;
        motionstudy.StopCalculateNotify += this.motionStudy_StopCalculateNotify;
    }

    public int motionStudy_ForceOutputTimeStepChangeNotify(double Time, object ForceNames, object
Disp, object Velocity, object Acceleration, object ForceOrTorque, ref object ForceTorqueValue)
    {
        // Fire event for every external force or torque output time step change
        MotionStudyProperties swMotionStudyProperties = default(MotionStudyProperties);
        int i = 0;
        double frameRate = 0;
        int numForces = 0;
        double deltaT = 0;
        double pi = 0;
        double w = 0;
        double A = 0;
        object[] arrForceNames;
        double[] arrForceTorqueValue;
        double[] arrAcceleration;

        arrForceNames = (object[])ForceNames;
        arrForceTorqueValue = (double[])ForceTorqueValue;
        arrAcceleration = (double[])Acceleration;

        pi = 3.14159265358979;
    }

```

```

        swMotionStudyProperties =
(MotionStudyProperties)motionstudy.GetProperties((int)swMotionStudyType_e.swMotionStudyTypeCosmosMotion);
        frameRate = swMotionStudyProperties.GetFrameRate();
        deltaT = 1 / frameRate;
        numForces = motionstudy.GetNumOfExternalForces();

        for (i = 0; i <= numForces - 1; i++)
        {
            A = (double)arrAcceleration[i];

            if ((arrForceNames[i].ToString() == "Force1") || (arrForceNames[i].ToString() ==
"Force2") || (arrForceNames[i].ToString() == "Force3"))
            {
                A = 0.8;
                w = 2 * pi / 1.5;
            }
            else if ((arrForceNames[i].ToString() == "Torque1" | arrForceNames[i].ToString() ==
"Torque2" | arrForceNames[i].ToString() == "Torque3"))
            {
                A = 0.005;
                w = pi;
            }
            else if ((arrForceNames[i].ToString() == "Action Reaction Force1" |
arrForceNames[i].ToString() == "Action Reaction Force2" | arrForceNames[i].ToString() == "Action Reaction
Force3"))
            {
                A = 0.002;
                w = 2 * pi;
            }
            else
            {
                A = 0;
                w = 0;
            }

            // Compute the return value and write to file
            arrForceTorqueValue[i] = A * System.Math.Sin(w * (Time + deltaT));
            writeOut(arrForceNames[i], arrForceTorqueValue[i]);
        }
        return 1;
    }

    public int motionStudy_MotorOutputTimeStepChangeNotify(double Time, object MotorNames, object
Position, object Velocity, object Acceleration, object ForceOrTorque, ref object MotorValue)
    {
        // Fire event for every external motor output time step change
        int numMotors = 0;
        MotionStudyProperties swMotionStudyProperties = default(MotionStudyProperties);
        double frameRate = 0;
        double deltaT = 0;
        double pi = 0;
        double w = 0;
        double A = 0;
        int i = 0;
        object[] arrMotorNames;
        double[] arrMotorValue;

        arrMotorNames = (object[])MotorNames;
        arrMotorValue = (double[])MotorValue;

        pi = 3.14159265358979;

        swMotionStudyProperties =
(MotionStudyProperties)motionstudy.GetProperties((int)swMotionStudyType_e.swMotionStudyTypeCosmosMotion);
        frameRate = swMotionStudyProperties.GetFrameRate();
        deltaT = 1 / frameRate;
        numMotors = motionstudy.GetNumOfExternalMotors();

        for (i = 0; i <= numMotors - 1; i++)
        {
            if ((arrMotorNames[i].ToString() == "RotaryMotor1") | (arrMotorNames[i].ToString() ==
"RotaryMotor2"))
            {

```

```

        A = (pi / 4);
        w = (pi / 2);
    }
    else if ((arrMotorNames[i].ToString() == "LinearMotor1"))
    {
        A = 0.05;
        w = 2 * pi / 3;
    }
    else if ((arrMotorNames[i].ToString() == "LinearMotor2"))
    {
        A = 0.025;
        w = 2 * pi;
    }
    else if ((arrMotorNames[i].ToString() == "LinearMotor3"))
    {
        A = 1.0472;
        w = 2 * pi;
    }
    else
    {
        A = 0;
        w = 0;
    }

    //Compute the return value and write to file
    arrMotorValue[i] = A * System.Math.Sin(w * (Time + deltaT));
    writeOut(arrMotorNames[i], arrMotorValue[i]);
}
return 1;
}

public int motionStudy_OutputTimeStepChangeNotify(double Time)
{
    // Fire event at every output step

    double myTime = 0;
    myTime = Time;
    return 1;
}

public int motionStudy_StartCalculateNotify(double Time)
{
    // Fire event when recalculation starts

    double myTime = 0;
    myTime = Time;

    System.IO.StreamWriter outputWrite = new
System.IO.StreamWriter("c:\\temp\\ForceAndMotorValuesCSharp.txt", true);
    outputWrite.WriteLine("Recalculation started...");
    outputWrite.WriteLine(" ");
    outputWrite.Close();

    return 1;
}

public int motionStudy_StopCalculateNotify(double Time)
{
    // Fire event when recalculation stops

    double myTime = 0;
    myTime = Time;

    System.IO.StreamWriter outputWrite = new
System.IO.StreamWriter("c:\\temp\\ForceAndMotorValuesCSharp.txt", true);
    outputWrite.WriteLine(" ");
    outputWrite.WriteLine("Recalculation stopped...");
    outputWrite.Close();

    return 1;
}

```

```
/// <summary>  
/// The SldWorks swApp variable is pre-assigned for you.  
/// </summary>
```

```
public SldWorks swApp;
```

```
}  
}
```

Fire Events for External Output Time Step Changes Example (VBA)

This example shows how to listen and catch events for external force and motor output time step changes.

```
'-----
' Preconditions:
' 1. Start SOLIDWORKS Design Premium, which includes SOLIDWORKS Motion.
' 2. Verify that the c:\temp folder exists.
' 3. In SOLIDWORKS Design:
'   a. Start the SOLIDWORKS Motion Study add-in (in
'      SOLIDWORKS Design, click Tools > Add-Ins > SOLIDWORKS Motion).
'   b. Open public_documents\samples\tutorial\api\simple-block.sldasm.
'   c. Click MotionStudy1 and select Motion Analysis in the
'      Type of Study list at the upper-left corner of the
'      MotionManager.
' 4. In the IDE:
'   a. Reference the SOLIDWORKS MotionStudy type library.
'   b. Copy and paste this code in the main module.
'   c. Click Insert > Class module and copy and paste this code in the
'      class module.
'
' Postconditions:
' 1. Calculates motion analysis.
' 2. Opens c:\temp\ForceAndMotorValuesVBA.txt for output.
' 3. Sets forces and motors to external.
' 4. Recalculates motion analysis.
' 5. Fires an event for every force and motor output time step change
'    and recalculates and writes the force magnitude or motor speed
'    to c:\temp\ForceAndMotorValuesVBA.txt.
' 6. Open and examine c:\temp\ForceAndMotorValuesVBA.txt.
'
' NOTE: Because this model is used elsewhere, do not save changes.
'-----
```

Main module

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swMotionMgr As swMotionStudy.MotionStudyManager
Dim swMotionStudy As swMotionStudy.motionStudy
Dim studyName As String
Dim numStudies As Long
Dim eventClass As Class1

Sub Main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    ' Motion study name
    studyName = "Motion Study 1"

    ' Get the MotionManager
    Set swMotionMgr = swModelDocExt.GetMotionStudyManager()
    numStudies = swMotionMgr.GetMotionStudyCount()
    Set swMotionStudy = swMotionMgr.GetMotionStudy(studyName)
    If Not swMotionStudy.IsActive Then swMotionStudy.Activate

    'Calculate the motion study
    swMotionStudy.Calculate

    ' Set up events
    Set eventClass = New Class1
    eventClass.Init swMotionStudy

    ' Open output file for writing force and motor values
    Open "c:\temp\ForceAndMotorValuesVBA.txt" For Output As #1

    ' Set feature Force1 as external force
    Call SetForceExternalFlag("Force1", True)
```

```

' Set feature LinearMotor1 as external motor
Call SetMotorExternalFlag("LinearMotor1", True)

' Fire event at every output time step
swMotionStudy.SetFireOutputTimeStepEvents (True)

'Recalculate the motion study
swMotionStudy.Calculate

' Close output file
Close #1

End Sub

Private Sub SetForceExternalFlag(ByVal featureName As String, ByVal setFlag As Long)
' Set forces to external; i.e., listen for events

Dim swFeature As SldWorks.Feature
Dim swSimulationFeatureData As SldWorks.SimulationForceFeatureData
Dim simTypeName As String
Dim motionFeatures As Variant
Dim ret As Boolean
Dim i As Long

motionFeatures = swMotionStudy.GetMotionFeatures
For i = 0 To UBound(motionFeatures)
Set swFeature = motionFeatures(i)
simTypeName = swFeature.GetTypeName
If simTypeName = "AEMLinearForce" Or simTypeName = "AEMTorque" Then
If swFeature.Name = featureName Then
Set swSimulationFeatureData = swFeature.GetDefinition
swSimulationFeatureData.ExternalState = setFlag
ret = swFeature.ModifyDefinition(swSimulationFeatureData, swModel, Nothing)
Write #1, "Feature: " & swFeature.Name
Write #1, " Listen for events: " & swSimulationFeatureData.ExternalState
Write #1, " "
End If
End If
Next i
End Sub

Private Sub SetMotorExternalFlag(ByVal featureName As String, ByVal setFlag As Long)
' Set motors to external; i.e., listen for events

Dim swFeature As SldWorks.Feature
Dim swSimulationFeatureData As SldWorks.SimulationMotorFeatureData
Dim simTypeName As String
Dim motionFeatures As Variant
Dim i As Long
Dim ret As Boolean

motionFeatures = swMotionStudy.GetMotionFeatures
For i = 0 To UBound(motionFeatures)
Set swFeature = motionFeatures(i)
simTypeName = swFeature.GetTypeName
If simTypeName = "AEMLinearMotor" Or simTypeName = "AEMRotationalMotor" Then
If swFeature.Name = featureName Then
Set swSimulationFeatureData = swFeature.GetDefinition
swSimulationFeatureData.ExternalState = setFlag
ret = swFeature.ModifyDefinition(swSimulationFeatureData, swModel, Nothing)
Write #1, "Feature name: " & swFeature.Name
Write #1, " Listen for events: " & swSimulationFeatureData.ExternalState
Write #1, " "
End If
End If
Next i
End Sub

```

[Back to top](#)

'Class module

```

Dim WithEvents motionStudy As swMotionStudy.motionStudy

Private Sub Class_Initialize()
End Sub

Sub Init(study As swMotionStudy.motionStudy)
Set motionStudy = study
End Sub

Private Function motionStudy_ForceOutputTimeStepChangeNotify(ByVal Time As Double, ByVal ForceNames As Variant, ByVal
Disp As Variant, ByVal Velocity As Variant, ByVal Acceleration As Variant, ByVal ForceOrTorque As Variant,

```

```

ForceTorqueValue As Variant) As Long
' Fire event for every external force or torque output time step change
Dim swMotionStudyProperties As swMotionStudy.motionStudyProperties
Dim frameRate As Double
Dim numForces As Long
Dim deltaT As Double
Dim pi As Double
Dim w As Double
Dim A As Double

pi = 3.14159265358979

Set swMotionStudyProperties = motionStudy.GetProperties(swMotionStudyTypeCosmosMotion)
frameRate = swMotionStudyProperties.GetFrameRate()
deltaT = 1 / frameRate
numForces = motionStudy.GetNumOfExternalForces

For i = 0 To numForces - 1
    A = Acceleration(i)

    If (ForceNames(i) = "Force1" Or ForceNames(i) = "Force2" Or ForceNames(i) = "Force3") Then
        Write #1, " " & ForceNames(i)
        A = 0.8
        w = 2 * pi / 1.5
    ElseIf (ForceNames(i) = "Torque1" Or ForceNames(i) = "Torque2" Or ForceNames(i) = "Torque3") Then
        Write #1, " " & ForceNames(i)
        A = 0.005
        w = pi
    ElseIf (ForceNames(i) = "Action Reaction Force1" Or ForceNames(i) = "Action Reaction Force2" Or ForceNames(i) =
"Action Reaction Force3") Then
        Write #1, " " & ForceNames(i)
        A = 0.002
        w = 2 * pi
    Else
        Write #1, " " & ForceNames(i)
        A = 0
        w = 0
    End If

    ' Compute the return value
    ForceTorqueValue(i) = A * Sin(w * (Time + deltaT))
    Write #1, "    Force or torque magnitude: " & ForceTorqueValue(i)

Next i

End Function

Private Function motionStudy_MotorOutputTimeStepChangeNotify(ByVal Time As Double, ByVal MotorNames As Variant, ByVal
Position As Variant, ByVal Velocity As Variant, ByVal Acceleration As Variant, ByVal ForceOrTorque As Variant,
MotorValue As Variant) As Long
' Fire event for every external motors at every output time step change
Dim numMotors as Long
Dim swMotionStudyProperties As swMotionStudy.motionStudyProperties
Dim frameRate As Double
Dim deltaT As Double
Dim pi As Double
Dim w As Double
Dim A As Double

pi = 3.14159265358979

Set swMotionStudyProperties = motionStudy.GetProperties(swMotionStudyTypeCosmosMotion)
frameRate = swMotionStudyProperties.GetFrameRate()
deltaT = 1 / frameRate
numMotors = motionStudy.GetNumOfExternalMotors

For i = 0 To numMotors - 1

    If (MotorNames(i) = "RotaryMotor1" Or MotorNames(i) = "RotaryMotor2") Then
        Write #1, " " & MotorNames(i)
        A = (pi / 4)
        w = (pi / 2)
    ElseIf (MotorNames(i) = "LinearMotor1") Then
        Write #1, " " & MotorNames(i)
        A = 0.05
        w = 2 * pi / 3
    ElseIf (MotorNames(i) = "LinearMotor2") Then
        Write #1, " " & MotorNames(i)
        A = 0.025
        w = 2 * pi
    ElseIf (MotorNames(i) = "LinearMotor3") Then
        Write #1, " " & MotorNames(i)

```

```

        A = 1.0472
        w = 2 * pi
    Else
        Write #1, "  LinearMotor"
        A = 0
        w = 0
    End If

    'Compute the return value
    MotorValue(i) = A * Sin(w * (Time + deltaT))
    Write #1, "    Motor speed: " & MotorValue(i)
Next i
End Function

Private Function motionStudy_OutputTimeStepChangeNotify(ByVal Time As Double) As Long
' Fire event at every output step
    myTime = Time
End Function

Private Function motionStudy_StartCalculateNotify(ByVal Time As Double) As Long
' Fire event when recalculation starts
    Write #1, "Recalculation started..."
    Write #1, " "
    myTime = Time
End Function

Private Function motionStudy_StopCalculateNotify(ByVal Time As Double) As Long
' Fire event when recalculation stops
    Write #1, " "
    Write #1, "Recalculation stopped..."
    myTime = Time
End Function

```

[Back to top](#)

Fire Events for External Output Time Step Changes Example (VB.NET)

This example shows how to listen and catch events for external force and motor output time step changes.

```
' -----
' Preconditions:
' 1. Start SOLIDWORKS Design Premium, which includes SOLIDWORKS Motion.
' 2. Verify that the c:\temp folder exists.
' 3. In SOLIDWORKS Design:
'   a. Start the SOLIDWORKS Motion Study add-in (in
'      SOLIDWORKS Design, click Tools > Add-Ins > SOLIDWORKS Motion).
'   b. Open public_documents\samples\tutorial\api\simple-block.sldasm.
'   c. Click MotionStudy 1 and select Motion Analysis in the
'      Type of Study list at the upper-left corner of the
'      MotionManager.
' 4. In the IDE, reference the SOLIDWORKS Motion Study interop
'   assembly (right-click the name of the project in the Project
'   Explorer > click Add Reference > browse to
'   install_dir\api\redist > click SolidWorks.Interop.swmotionstudy).
'
' Postconditions:
' 1. Calculates motion analysis.
' 2. Opens c:\temp\ForceAndMotorValuesVBNET.txt for output.
' 3. Sets forces and motors to external.
' 4. Recalculates motion analysis.
' 5. Fires an event for every force and motor output time step change
'   and recalculates and writes the force magnitude or motor speed to
'   c:\temp\ForceAndMotorValuesVBNET.txt.
' 6. Open and examine c:\temp\ForceAndMotorValuesVBNET.txt.
'
' NOTE: Because this model is used elsewhere, do not save changes.
' -----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swmotionstudy
```

```
Partial Class SolidWorksMacro
```

```
    Public WithEvents motionstudy As MotionStudy
    Public swMotionStudy As MotionStudy
    Public swModel As ModelDoc2
    Public outputWrite As System.IO.StreamWriter

    Public Sub Main()

        Dim swModelDocExt As ModelDocExtension
        Dim swMotionMgr As MotionStudyManager
        Dim studyName As String
        Dim numStudies As Integer

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        ' Motion study name
        studyName = "Motion Study 1"

        ' Get the MotionManager
        swMotionMgr = swModelDocExt.GetMotionStudyManager()
        numStudies = swMotionMgr.GetMotionStudyCount()
        swMotionStudy = swMotionMgr.GetMotionStudy(studyName)
        If Not swMotionStudy.IsActive Then swMotionStudy.Activate()

        'Calculate the motion study
        swMotionStudy.Calculate()

        ' Set up events
        motionstudy = swMotionStudy
```

```
AttachEventHandlers()
```

```
' Open output file for writing force and motor values  
outputWrite = IO.File.CreateText("c:\temp\ForceAndMotorValuesVBNET.txt")
```

```
' Set feature Force1 as external force  
Call SetForceExternalFlag("Force1", True)
```

```
' Set feature LinearMotor1 as external motor  
Call SetMotorExternalFlag("LinearMotor1", True)
```

```
' Fire event at every output time step  
swMotionStudy.SetFireOutputTimeStepEvents(True)
```

```
'Recalculate the motion study  
swMotionStudy.Calculate()
```

```
' Close output file  
outputWrite.Close()
```

```
End Sub
```

```
Private Sub SetForceExternalFlag(ByVal featureName As String, ByVal setFlag As Integer)  
' Set forces to external; i.e., listen for events
```

```
Dim swFeature As Feature  
Dim swSimulationFeatureData As SimulationForceFeatureData  
Dim simTypeName As String  
Dim motionFeatures As Object  
Dim ret As Boolean  
Dim i As Integer
```

```
motionFeatures = swMotionStudy.GetMotionFeatures
```

```
For i = 0 To UBound(motionFeatures)
```

```
swFeature = motionFeatures(i)
```

```
simTypeName = swFeature.GetTypeName
```

```
If simTypeName = "AEMLinearForce" Or simTypeName = "AEMTorque" Then
```

```
    If swFeature.Name = featureName Then
```

```
        swSimulationFeatureData = swFeature.GetDefinition
```

```
        swSimulationFeatureData.ExternalState = setFlag
```

```
        ret = swFeature.ModifyDefinition(swSimulationFeatureData, swModel, Nothing)
```

```
        outputWrite.WriteLine("Feature: " & swFeature.Name)
```

```
        outputWrite.WriteLine(" Listen for events: " &
```

```
swSimulationFeatureData.ExternalState)
```

```
        outputWrite.WriteLine(" ")
```

```
    End If
```

```
End If
```

```
Next i
```

```
End Sub
```

```
Private Sub SetMotorExternalFlag(ByVal featureName As String, ByVal setFlag As Integer)  
' Set motors to external; i.e., listen for events
```

```
Dim swFeature As Feature  
Dim swSimulationFeatureData As SimulationMotorFeatureData  
Dim simTypeName As String  
Dim motionFeatures As Object  
Dim i As Integer  
Dim ret As Boolean
```

```
motionFeatures = swMotionStudy.GetMotionFeatures
```

```
For i = 0 To UBound(motionFeatures)
```

```
swFeature = motionFeatures(i)
```

```
simTypeName = swFeature.GetTypeName
```

```
If simTypeName = "AEMLinearMotor" Or simTypeName = "AEMRotationalMotor" Then
```

```
    If swFeature.Name = featureName Then
```

```
        swSimulationFeatureData = swFeature.GetDefinition
```

```
        swSimulationFeatureData.ExternalState = setFlag
```

```
        ret = swFeature.ModifyDefinition(swSimulationFeatureData, swModel, Nothing)
```

```
        outputWrite.WriteLine("Feature name: " & swFeature.Name)
```

```
        outputWrite.WriteLine(" Listen for events: " &
```

```
swSimulationFeatureData.ExternalState)
```

```
        outputWrite.WriteLine(" ")
```

```

        End If
    End If
Next i
End Sub

Sub AttachEventHandlers()
    AttachSWEvents()
End Sub

Sub AttachSWEvents()
    AddHandler motionstudy.ForceOutputTimeStepChangeNotify, AddressOf
Me.motionStudy_ForceOutputTimeStepChangeNotify
    AddHandler motionstudy.MotorOutputTimeStepChangeNotify, AddressOf
Me.motionStudy_MotorOutputTimeStepChangeNotify
    AddHandler motionstudy.OutputTimeStepChangeNotify, AddressOf
Me.motionStudy_OutputTimeStepChangeNotify
    AddHandler motionstudy.StartCalculateNotify, AddressOf Me.motionStudy_StartCalculateNotify
    AddHandler motionstudy.StopCalculateNotify, AddressOf Me.motionStudy_StopCalculateNotify
End Sub

Private Function motionStudy_ForceOutputTimeStepChangeNotify(ByVal Time As Double, ByVal ForceNames
As Object, ByVal Disp As Object, ByVal Velocity As Object, ByVal Acceleration As Object, ByVal
ForceOrTorque As Object, ByRef ForceTorqueValue As Object) As Integer
    ' Fire event for every external force or torque output time step change
    Dim swMotionStudyProperties As MotionStudyProperties
    Dim i As Integer
    Dim frameRate As Double
    Dim numForces As Integer
    Dim deltaT As Double
    Dim pi As Double
    Dim w As Double
    Dim A As Double

    pi = 3.14159265358979

    swMotionStudyProperties =
motionstudy.GetProperties(swMotionStudyType_e.swMotionStudyTypeCosmosMotion)
    frameRate = swMotionStudyProperties.GetFrameRate()
    deltaT = 1 / frameRate
    numForces = motionstudy.GetNumOfExternalForces

    For i = 0 To numForces - 1

        A = Acceleration(i)

        If (ForceNames(i) = "Force1" Or ForceNames(i) = "Force2" Or ForceNames(i) = "Force3") Then
            outputWrite.WriteLine(" " & ForceNames(i))
            A = 0.8
            w = 2 * pi / 1.5
        ElseIf (ForceNames(i) = "Torque1" Or ForceNames(i) = "Torque2" Or ForceNames(i) = "Torque3")
Then
            outputWrite.WriteLine(" " & ForceNames(i))
            A = 0.005
            w = pi
        ElseIf (ForceNames(i) = "Action Reaction Force1" Or ForceNames(i) = "Action Reaction Force2"
Or ForceNames(i) = "Action Reaction Force3") Then
            outputWrite.WriteLine(" " & ForceNames(i))
            A = 0.002
            w = 2 * pi
        Else
            outputWrite.WriteLine(" " & ForceNames(i))
            A = 0
            w = 0
        End If

        ' Compute the return value
        ForceTorqueValue(i) = A * Math.Sin(w * (Time + deltaT))
        outputWrite.WriteLine("    Force or torque magnitude: " & ForceTorqueValue(i))

    Next i

End Function

Private Function motionStudy_MotorOutputTimeStepChangeNotify(ByVal Time As Double, ByVal MotorNames

```

```

As Object, ByVal Position As Object, ByVal Velocity As Object, ByVal Acceleration As Object, ByVal
ForceOrTorque As Object, ByRef MotorValue As Object) As Integer
' Fire event for every external motor output time step change
Dim numMotors As Integer
Dim swMotionStudyProperties As MotionStudyProperties
Dim frameRate As Double
Dim deltaT As Double
Dim pi As Double
Dim w As Double
Dim A As Double
Dim i As Integer

pi = 3.14159265358979

swMotionStudyProperties =
motionstudy.GetProperties(swMotionStudyType_e.swMotionStudyTypeCosmosMotion)
frameRate = swMotionStudyProperties.GetFrameRate()
deltaT = 1 / frameRate
numMotors = motionstudy.GetNumOfExternalMotors

For i = 0 To numMotors - 1

    If (MotorNames(i) = "RotaryMotor1" Or MotorNames(i) = "RotaryMotor2") Then
        outputWrite.WriteLine(" " & MotorNames(i))
        A = (pi / 4)
        w = (pi / 2)
    ElseIf (MotorNames(i) = "LinearMotor1") Then
        outputWrite.WriteLine(" " & MotorNames(i))
        A = 0.05
        w = 2 * pi / 3
    ElseIf (MotorNames(i) = "LinearMotor2") Then
        outputWrite.WriteLine(" " & MotorNames(i))
        A = 0.025
        w = 2 * pi
    ElseIf (MotorNames(i) = "LinearMotor3") Then
        outputWrite.WriteLine(" " & MotorNames(i))
        A = 1.0472
        w = 2 * pi
    Else
        outputWrite.WriteLine(" LinearMotor")
        A = 0
        w = 0
    End If

    'Compute the return value
    MotorValue(i) = A * Math.Sin(w * (Time + deltaT))
    outputWrite.WriteLine(" Motor speed: " & MotorValue(i))
Next i
End Function

Private Function motionStudy_OutputTimeStepChangeNotify(ByVal Time As Double) As Integer
' Fire event at every output step

Dim myTime As Double

myTime = Time

End Function

Private Function motionStudy_StartCalculateNotify(ByVal Time As Double) As Integer
' Fire event when recalculation starts

Dim myTime As Double

outputWrite.WriteLine("Recalculation started...")
outputWrite.WriteLine(" ")
myTime = Time

End Function

Private Function motionStudy_StopCalculateNotify(ByVal Time As Double) As Integer
' Fire event when recalculation stops

Dim myTime As Double

```

```
outputWrite.WriteLine(" ")  
outputWrite.WriteLine("Recalculation stopped...")
```

```
myTime = Time
```

```
End Function
```

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>
```

```
Public swApp As SldWorks
```

```
End Class
```

Fire Events When Display State Changes Example (C#)

This example shows how to fire the events related to changing display states of a configuration in an assembly document.

```
//-----  
// Preconditions:  
// 1. Open an assembly document that has a configuration with multiple  
//    display states.  
// 2. Run this macro in debug mode.  
// 3. Change the display state of the active configuration in SOLIDWORKS Design  
//    (click the ConfigurationManager tab and double-click  
//    a different display state).  
//  
// Postconditions:  
// 1. Displays a message box informing you that the display state is about to  
//    change.  
// 2. After the display state changes, displays another message informing you  
//    that the display state has changed.  
//  
// NOTE: This example also fires these events when you change  
// configurations in an assembly document.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Collections;  
using System.Windows.Forms;  
  
namespace Macro1.csproj  
{  
    public  
    partial class SolidWorksMacro  
    {  
        public AssemblyDoc swAssem;  
        public void Main()  
        {  
            ModelDoc2 swModel;  
            Hashtable openAssem;  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
  
            //Set up events  
            swAssem = (AssemblyDoc)swModel;  
            openAssem = new Hashtable();  
            AttachEventHandlers();  
        }  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
        public SldWorks swApp;  
  
        public void AttachEventHandlers()  
        {  
            AttachSWEvents();  
        }  
  
        public void AttachSWEvents()  
        {  
            swAssem.ActiveDisplayStateChangePreNotify +=  
this.swAssem.ActiveDisplayStateChangePreNotify;  
            swAssem.ActiveDisplayStateChangePostNotify +=  
this.swAssem.ActiveDisplayStateChangePostNotify;  
        }  
    }  
}
```

```
public int swAssem_ActiveDisplayStateChangePreNotify()
//Send message when user changes display state in the ConfigurationManager
{
    MessageBox.Show("The active configuration's display state is about to change.");
    return 1;
}

public int swAssem_ActiveDisplayStateChangePostNotify(string DisplayStateName)
//Send message after user changes display state in the ConfigurationManager
{
    MessageBox.Show("The active configuration's display state has changed.");
    return 1;
}
}
```

Fire Events When Display State Changes Example (VBA)

This example shows how to fire the events related to changing display states of a configuration in an assembly document.

```
'-----
' Preconditions:
' 1. Copy Main module into your macro's main module.
' 2. Click Insert > Class module and copy Class module into that module.
' 3. Open an assembly document that has a configuration with multiple
'    display states.
'    a. Run this macro in debug mode.
'    b. Change the display state of the active configuration in SOLIDWORKS Design
'       (click the ConfigurationManager tab and double-click
'        a different display state).
'
' Postconditions:
' 1. Displays a message box informing you that the display state is about to
'    change.
' 2. After the display state changes, displays another message informing you
'    that the display state has changed.
'
' NOTE: This example also fires these events when you change
' configurations in an assembly document.
'-----
'Main module
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssem As SldWorks.AssemblyDoc
Dim swAssemEvents As Class1

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    'Set up events
    Set swAssem = swModel
    Set swAssemEvents = New Class1
    Set swAssemEvents.swAssem = swApp.ActiveDoc

End Sub

'Class module
Option Explicit

Public WithEvents swAssem As SldWorks.AssemblyDoc

'Send message when user changes display state in the ConfigurationManager
Private Function swAssem_ActiveDisplayStateChangePreNotify() As Long
    MsgBox "The active configuration's display state is about to change."
End Function

'Send message after user changes display state in the ConfigurationManager
Private Function swAssem_ActiveDisplayStateChangePostNotify(ByVal DisplayStateName As String) As Long
    MsgBox "The active configuration's display state has changed."
End Function
```

[Back to top](#)

Fire Events When Display State Changes Example (VB.NET)

This example shows how to fire the events related to changing display states of a configuration in an assembly document.

```
'-----
' Preconditions:
' 1. Open an assembly document that has a configuration with multiple
'    display states.
' 2. Run this macro in debug mode.
' 3. Change the display state of the active configuration in SOLIDWORKS Design
'    (click the ConfigurationManager tab and double-click
'    a different display state).
'
' Postconditions:
' 1. Displays a message box informing you that the display state is about to
'    change.
' 2. After the display state changes, displays another message informing you
'    that the display state has changed.
'
' NOTE: This example also fires these events when you change
' configurations in an assembly document.
'-----
Option Explicit On

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
Imports System.Collections

Partial Class SolidWorksMacro
    Public WithEvents swAssem As AssemblyDoc

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim openAssem As Hashtable
        swModel = swApp.ActiveDoc

        'Set up events
        swAssem = swModel
        openAssem = New Hashtable
        AttachEventHandlers()

    End Sub

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub

    Sub AttachSWEvents()
        AddHandler swAssem.ActiveDisplayStateChangePreNotify, AddressOf
Me.swAssem_ActiveDisplayStateChangePreNotify
        AddHandler swAssem.ActiveDisplayStateChangePostNotify, AddressOf
Me.swAssem_ActiveDisplayStateChangePostNotify
    End Sub

    Private Function swAssem_ActiveDisplayStateChangePreNotify() As Integer
        'Send message when user changes display state in the ConfigurationManager
        MsgBox("The active configuration's display state is about to change.")
    End Function
```

```
Private Function swAssem_ActiveDisplayStateChangePostNotify(ByVal DisplayStateName As String) As Integer
    'Send message after user changes display state in the ConfigurationManager
    MsgBox("The active configuration's display state has changed.")
End Function

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class
```

Fire Events When Display State Changes in Part Document (C#)

This example shows how to fire the events related to changing display states of a configuration in a part document.

```
//-----  
// Preconditions:  
// 1. Open a part document that has  
//    a configuration with multiple display states.  
// 2. Run this macro in debug mode.  
// 3. Change the display state of the active configuration in SOLIDWORKS Design  
//    (click the ConfigurationManager tab and double-click  
//    a different display state).  
//  
// Postcondition: A message box is displayed informing  
// you that the display state is about change. After the display state changes,  
// another message box is displayed informing you that the display  
// state has changed.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Collections;  
using System.Windows.Forms;  
namespace FireEventsWhenDisplayStateChangesCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
        public PartDoc swPart;  
        public void Main()  
        {  
            ModelDoc2 swModel;  
            Hashtable openPart;  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            //Set up events  
            swPart = (PartDoc)swModel;  
            openPart = new Hashtable();  
            AttachEventHandlers();  
        }  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
        public SldWorks swApp;  
        public void AttachEventHandlers()  
        {  
            AttachSWEvents();  
        }  
    }  
}
```

```

public void AttachSWEvents()
{
    swPart.ActiveDisplayStateChangePreNotify += this.swPart_ActiveDisplayStateChangePreNotify;
    swPart.ActiveDisplayStateChangePostNotify +=
this.swPart_ActiveDisplayStateChangePostNotify;
}
public int swPart_ActiveDisplayStateChangePreNotify()
//Send message when user changes display state in the ConfigurationManager
{
    MessageBox.Show("The active configuration's display state is about to change.");
    return 1;
}
public int swPart_ActiveDisplayStateChangePostNotify(string DisplayStateName)
//Send message after user changes display state in the ConfigurationManager
{
    MessageBox.Show("The active configuration's display state has changed.");
    return 1;
}
}
}

```

Fire Events When Display State Changes in Part Document Example (VBA)

This example shows how to fire the events related to changing display states of a configuration in a part document.

- [Module](#)
- [Class Module](#)

Module

```
'-----  
' Preconditions:  
' 1. Open a part document that has  
'    a configuration with multiple display states.  
' 2. Run this macro in debug mode.  
' 3. Change the display state of the active configuration in SOLIDWORKS Design  
'    (click the ConfigurationManager tab and double-click  
'    a different display state).  
'  
' Postcondition: A message box is displayed informing  
' you that the display state is about change. After the display state changes,  
' another message box is displayed informing you that the display  
' state has changed.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swPart As SldWorks.PartDoc  
Dim swPartEvents As Class1
```

```
Sub main()  
    Set swApp = Application.SldWorks  
    Set swPart = swApp.ActiveDoc  
  
    'Set up events  
    Set swPartEvents = New Class1  
    Set swPartEvents.swPart = swApp.ActiveDoc  
End Sub
```

Class Module

Option Explicit

```
Public WithEvents swPart As SldWorks.PartDoc
```

```
Private Function swPart_ActiveDisplayStateChangePreNotify() As Long  
'Send message when user changes display state in the ConfigurationManager
```

```
    MsgBox "The active configuration's display state is about to change."  
End Function
```

```
Private Function swPart_ActiveDisplayStateChangePostNotify(ByVal DisplayStateName As String) As Long  
'Send message after user changes display state in the ConfigurationManager  
    MsgBox "The active configuration's display state has changed."  
End Function
```

Fire Events When Display State Changes in Part Document Example (VB.NET)

This example shows how to fire the events related to changing display states of a configuration in a part document.

```
' -----  
' Preconditions:  
' 1. Open a part document that has  
'    a configuration with multiple display states.  
' 2. Run this macro in debug mode.  
' 3. Change the display state of the active configuration in SOLIDWORKS Design  
'    (click the ConfigurationManager tab and double-click  
'    a different display state).  
'  
' Postcondition: A message box is displayed informing  
' you that the display state is about change. After the display state changes,  
' another message box is displayed informing you that the display  
' state has changed.  
' -----
```

Option Explicit On

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics  
Imports System.Collections
```

Partial Class SolidWorksMacro

Public WithEvents swPart As PartDoc

Public Sub Main()

Dim swModel As ModelDoc2
Dim openPart As Hashtable

swModel = swApp.ActiveDoc

'Set up events
swPart = swModel
openPart = New Hashtable
AttachEventHandlers()

End Sub

Sub AttachEventHandlers()
AttachSWEvents()

End Sub

```

Sub AttachSWEvents()
    AddHandler swPart.ActiveDisplayStateChangePreNotify, AddressOf
Me.swPart_ActiveDisplayStateChangePreNotify
    AddHandler swPart.ActiveDisplayStateChangePostNotify, AddressOf
Me.swPart_ActiveDisplayStateChangePostNotify
End Sub

Private Function swPart_ActiveDisplayStateChangePreNotify() As Integer
    'Send message when user changes display state in the ConfigurationManager
    MsgBox("The active configuration's display state is about to change.")
End Function

Private Function swPart_ActiveDisplayStateChangePostNotify(ByVal DisplayStateName As String) As
Integer
    'Send message after user changes display state in the ConfigurationManager
    MsgBox("The active configuration's display state has changed.")
End Function

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Fire Events When Dragging Instant3D Manipulator in a Part Example (C#)

This example shows how to fire events when dragging an Instant3D manipulator in a part document.

```
//-----  
// Preconditions:  
// 1. Open a part document.  
// 2. Open the Immediate window.  
//  
// NOTE: Instant3D is enabled by the macro.  
//  
// Postconditions:  
// 1. Select an Instant3D manipulator in the  
//    open part. For example,  
//    double-click an extrude feature in a part,  
//    then select one of the Instant3D manipulators  
//    and drag it.  
// 2. Writes a debug statement to the Immediate  
//    window informing you that dragging of an  
//    Instant3D manipulator has started.  
// 3. Stop dragging the manipulator.  
// 4. Writes a debug statement to the Immediate  
//    window informing you that dragging of  
//    an Instant3D manipulator has stopped.  
// 5. Examine the Immediate window.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Collections;  
using System.Diagnostics;  
namespace DragStateChangeNotifyPartCSharp.csproj  
{  
    partial class SolidWorksMacro    {  
        public PartDoc pDoc;  
        public FeatureManager swFeatMgr;  
        public void Main()  
        {  
            ModelDoc2 swModel;  
            Hashtable openPart;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
  
            //Enable Instant3D manipulators  
            swFeatMgr = (FeatureManager)swModel.FeatureManager;  
            swFeatMgr.MoveSizeFeatures = true;  
  
            // Set up event handler  
            pDoc = (PartDoc)swModel;  
            openPart = new Hashtable();  
  
            AttachEventHandlers();  
        }  
  
        public void AttachEventHandlers()  
        {  
            AttachSWEvents();  
        }  
  
        public void AttachSWEvents()  
        {  
            if ((pDoc != null))  
            {  
                pDoc.DragStateChangeNotify += this.pDoc_DragStateChangeNotify;
```

```

    }
}

private int pDoc_DragStateChangeNotify(bool State)
{
    int functionReturnValue = 0;
    // Write debug statement when dragging of manipulator started and stopped
    if (State == true)
    {
        Debug.Print("Dragging of manipulator started.");
        functionReturnValue = 1;
    }
    else
    {
        Debug.Print("Dragging of manipulator stopped.");
        functionReturnValue = 0;
    }
    return functionReturnValue;
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Fire Events When Dragging Instant3D Manipulator in a Part Example (VBA)

This example shows how to fire events when dragging an Instant3D manipulator in a part document.

```
' -----  
' Preconditions:  
' 1. Open a part document.  
' 2. Copy and paste Main into the main module.  
' 3. Click Insert > Class module and copy and paste  
' Class into the class module.  
' 4. Open the Immediate window.  
'  
' NOTE: Instant3D is enabled by the macro.  
'  
' Postconditions:  
' 1. Select an Instant3D manipulator in the  
' open part. For example,  
' double-click an extrude feature in a part,  
' then select one of the Instant3D manipulators  
' and drag it.  
' 2. Writes a debug statement to the Immediate window  
' informing you that dragging of an Instant3D  
' manipulator has started.  
' 3. Stop dragging the manipulator.  
' 4. Writes a debug statement to the Immediate  
' window informing you that dragging of  
' an Instant3D manipulator has stopped.  
' 5. Examine the Immediate window.  
' -----  
' Main
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swFeatMgr As SldWorks.FeatureManager  
Dim partDoc As New Class1
```

Sub main()

```
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc
```

```
' Enable Instant3D  
Set swFeatMgr = swModel.FeatureManager  
swFeatMgr.MoveSizeFeatures = True
```

```
' Execute its class module  
partDoc.init swModel
```

End Sub

[Back to top](#)

' Class

```
Public WithEvents doc As partDoc
```

```
Public Function init(ByRef docIn As Object)  
Set doc = docIn  
End Function
```

```
Private Function doc_DragStateChangeNotify(ByVal State As Boolean) As Long  
' Write debug statement when dragging of manipulator started and stopped  
If State = True Then  
Debug.Print "Dragging of manipulator started."  
Else  
Debug.Print "Dragging of manipulator stopped."
```

End If
End Function

[Back to top](#)

Fire Events When Dragging Instant3D Manipulator in a Part Example (VB.NET)

This example shows how to fire events when dragging an Instant3D manipulator in a part document.

```
' -----  
' Preconditions:  
' 1. Open a part document.  
' 2. Open the Immediate window.  
'  
' NOTE: Instant3D is enabled by the macro.  
'  
' Postconditions:  
' 1. Select an Instant3D manipulator in the  
'    open part. For example,  
'    double-click an extrude feature in a part,  
'    then select one of the Instant3D manipulators  
'    and drag it.  
' 2. Writes a debug statement to the Immediate window  
'    informing you that dragging of an Instant3D  
'    manipulator has started.  
' 3. Stop dragging the manipulator.  
' 4. Writes a debug statement to the Immediate  
'    window informing you that dragging of  
'    an Instant3D manipulator has stopped.  
' 5. Examine the Immediate window.  
' -----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Collections  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Public WithEvents pDoc As PartDoc  
  
    Public Sub main()  
  
        Dim swModel As ModelDoc2  
        Dim swFeatMgr As FeatureManager  
        Dim openPart As Hashtable  
  
        swModel = swApp.ActiveDoc  
  
        ' Enable Instant3D  
        swFeatMgr = swModel.FeatureManager  
        swFeatMgr.MoveSizeFeatures = True  
  
        ' Execute events  
        pDoc = swModel  
        openPart = New Hashtable  
        AttachEventHandlers()  
  
    End Sub  
  
    Sub AttachEventHandlers()  
        AttachSWEvents()  
    End Sub  
  
    Sub AttachSWEvents()  
        If Not pDoc Is Nothing Then  
            AddHandler pDoc.DragStateChangeNotify, AddressOf Me.pDoc_DragStateChangeNotify
```

```
End If
End Sub
```

```
Private Function pDoc_DragStateChangeNotify(ByVal State As Boolean) As Integer
    ' Write debug statement when dragging of manipulator started and stopped
    If State = True Then
        Debug.Print("Dragging of manipulator started.")
    Else
        Debug.Print("Dragging of manipulator stopped.")
    End If
End Function
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Fire Events When Dragging Instant3D Manipulator in an Assembly Example (C#)

This example shows how to fire events when dragging an Instant3D manipulator in an assembly document.

```
//-----  
// Preconditions:  
// 1. Open an assembly document.  
// 2. Open the Immediate window.  
//  
// NOTE: Instant3D is enabled by the macro.  
//  
// Postconditions:  
// 1. Select an Instant3D manipulator in the  
//    open assembly by right-clicking a floating  
//    assembly component, clicking the down arrows  
//    at the bottom of the shortcut menu, and selecting  
//    Move with Triad.  
// 2. Drag the Instant3D handle to move the assembly component.  
// 3. Writes a debug statement to the Immediate window  
//    informing you that dragging of an Instant3D manipulator  
//    has started.  
// 4. Stop dragging the manipulator.  
// 5. Writes a debug statement to the Immediate  
//    window informing you that dragging of  
//    an Instant3D manipulator has stopped.  
// 6. Examine the Immediate window.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Collections;  
using System.Diagnostics;  
namespace DragStateChangeNotifyAssemblyCSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public AssemblyDoc pDoc;  
        public FeatureManager swFeatMgr;  
        public void Main()  
        {  
            ModelDoc2 swModel;  
            Hashtable openAssembly;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
  
            //Enable Instant3D manipulators  
            swFeatMgr = (FeatureManager)swModel.FeatureManager;  
            swFeatMgr.MoveSizeFeatures = true;  
  
            // Set up event handler  
            pDoc = (AssemblyDoc)swModel;  
            openAssembly = new Hashtable();  
            AttachEventHandlers();  
        }  
  
        public void AttachEventHandlers()  
        {  
            AttachSWEvents();  
        }  
  
        public void AttachSWEvents()  
        {  
            if ((pDoc != null))  
            {  
                pDoc.DragStateChangeNotify += this.pDoc_DragStateChangeNotify;
```

```

    }
}

private int pDoc_DragStateChangeNotify(bool State)
{
    int functionReturnValue = 0;
    // Write debug statement when dragging of manipulator started and stopped
    if (State == true)
    {
        Debug.Print("Dragging of manipulator started.");
        functionReturnValue = 1;
    }
    else
    {
        Debug.Print("Dragging of manipulator stopped.");
        functionReturnValue = 0;
    }
    return functionReturnValue;
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Fire Events When Dragging Instant3D Manipulator in an Assembly Example (VBA)

This example shows how to fire events when dragging an Instant3D manipulator in an assembly document.

```
' -----  
' Preconditions:  
' 1. Open an assembly document.  
' 2. Copy and paste Main into the main module.  
' 3. Click Insert > Class module and copy and paste  
' Class1 into the class module.  
' 4. Open the Immediate window.  
'  
' NOTE: Instant3D is enabled by the macro.  
'  
' Postconditions:  
' 1. Select an Instant3D manipulator in the  
' open assembly by right-clicking a floating  
' assembly component, clicking the down arrows  
' at the bottom of the shortcut menu, and selecting  
' Move with Triad.  
' 2. Drag the Instant3D handle to move the assembly  
' component.  
' 3. Writes a debug statement to the Immediate window  
' informing you that dragging of an Instant3D  
' manipulator has started.  
' 4. Stop dragging the manipulator.  
' 5. Writes a debug statement to the Immediate window  
' informing you that dragging of an Instant3D  
' manipulator has stopped.  
' 6. Examine the Immediate window.  
' -----  
' Main
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swFeatMgr As SldWorks.FeatureManager  
Dim assemblyDoc As New Class1  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    ' Enable Instant3D  
    Set swFeatMgr = swModel.FeatureManager  
    swFeatMgr.MoveSizeFeatures = True  
  
    ' Execute its class module  
    assemblyDoc.init swModel  
End Sub
```

[Back to top](#)

```
' Class  
Public WithEvents doc As assemblyDoc  
  
Public Function init(ByRef docIn As Object)  
    Set doc = docIn  
End Function
```

```
Private Function doc_DragStateChangeNotify(ByVal State As Boolean) As Long
    ' Write debug statement when dragging of manipulator started and stopped
    If State = True Then
        Debug.Print "Dragging of manipulator started."
    Else
        Debug.Print "Dragging of manipulator stopped."
    End If
End Function
```

[Back to top](#)

Fire Events When Dragging Instant3D Manipulator in an Assembly Example (VB.NET)

This example shows how to fire events when dragging an Instant3D manipulator in an assembly document.

```
' -----  
' Preconditions:  
' 1. Open an assembly document.  
' 2. Open the Immediate window.  
'  
' NOTE: Instant3D is enabled by the macro.  
'  
' Postconditions:  
' 1. Select an Instant3D manipulator in the  
'    open assembly by right-clicking a floating  
'    assembly component, clicking the down arrows  
'    at the bottom of the shortcut menu, and selecting  
'    Move with Triad.  
' 2. Drag the Instant3D handle to move the assembly  
'    component.  
' 3. Writes a debug statement to the Immediate window  
'    informing you that dragging of an Instant3D  
'    manipulator has started.  
' 4. Stop dragging the manipulator.  
' 5. Writes a debug statement to the Immediate window  
'    informing you that dragging of an Instant3D  
'    manipulator has stopped.  
' 6. Examine the Immediate window.  
' -----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Collections  
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

Public WithEvents pDoc As AssemblyDoc

Public Sub main()

Dim swModel As ModelDoc2
Dim swFeatMgr As FeatureManager
Dim openAssembly As Hashtable

swModel = swApp.ActiveDoc

' Enable Instant3D
swFeatMgr = swModel.FeatureManager
swFeatMgr.MoveSizeFeatures = True

' Execute events
pDoc = swModel
openAssembly = New Hashtable

AttachEventHandlers()

End Sub

Sub AttachEventHandlers()
AttachSWEvents()
End Sub

```
Sub AttachSWEvents()  
    If Not pDoc Is Nothing Then  
        AddHandler pDoc.DragStateChangeNotify, AddressOf Me.pDoc_DragStateChangeNotify  
    End If  
End Sub
```

```
Private Function pDoc_DragStateChangeNotify(ByVal State As Boolean) As Integer  
    ' Write debug statement when dragging of manipulator started and stopped  
    If State = True Then  
        Debug.Print("Dragging of manipulator started.")  
    Else  
        Debug.Print("Dragging of manipulator stopped.")  
    End If  
End Function
```

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```

Fire Events When PropertyManager Page Opened and Canceled (C#)

This example shows how to get the SOLIDWORKS command ID, PropertyManager title, and whether the user interface is active. Events are fired before the PropertyManager page is opened and when it is canceled.

```
//-----
// Preconditions:
// 1. Verify that the part to open exists.
// 2. Add a reference to SolidWorks.Interop.swcommands.dll.
// 3. Open the Immediate window.
// 4. Clear Tools > Options > Stop VSTA debugger on
//    macro exit, if it's selected.
//
// Postconditions:
// 1. Opens the part.
// 2. Fires the CommandOpenPreNotify event; click OK
//    to close the message box.
// 3. Opens the Fillet PropertyManager page.
// 4. Gets the title of the PropertyManager page, whether the
//    user interface is active, and whether the command ID
//    is a fillet.
// 5. Click X on the Fillet PropertyManager page
//    to cancel it.
// 6. Fires the CommandCloseNotify event; click OK to close
//    the message box.
// 7. Examine the Immediate window.
// 8. Click Stop Debugging in the IDE.
// 9. Select Tools > Options > Stop VSTA debugger on
//    macro exit, if you cleared it in Preconditions
//    step 4.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using SolidWorks.Interop.swcommands;
using System.Windows.Forms;

namespace GetRunningCommandInfoSldWorksCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public SldWorks swAppSW;

        public void Main()
        {
            ModelDoc2 swModel;
            ModelDocExtension swModelDocExt;
            string modelName = null;
            int errors = 0;
            int warnings = 0;
            int commandID = 0;
            string pmpTitle = null;
            bool isUIActive = false;

            // Set up event
            swAppSW = (SldWorks)swApp;
            AttachEventHandlers();

            // Open the model
            modelName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\fillets\\knob.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(modelName, (int)swDocumentTypes_e.swDocPART,
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
```

```

        // Open the Fillet PropertyManager Page,
        // which causes the CommandOpenPreNotify event
        // to fire
        swModelDocExt.RunCommand((int)swCommands_e.swCommands_Fillet, "Fillet");

        // Get the command ID if the command is a fillet,
        // get the PropertyManager page title if one is active,
        // and get whether the user interface is active
        swApp.GetRunningCommandInfo(out commandID, out pmpTitle, out isUIActive);

        if (!string.IsNullOrEmpty(pmpTitle)) Debug.Print("Title of PropertyManager page: " +
pmpTitle);
        Debug.Print("Is user interface active? " + isUIActive);
        if ((commandID == 9))
        {
            Debug.Print("Command ID: " + "swCommands_Fillet");
        }
        else
        {
            Debug.Print("Command ID: " + "Not a fillet.");
        }
    }

    public void AttachEventHandlers()
    {
        AttachSWEvents();
    }

    public void AttachSWEvents()
    {
        swAppSW.CommandOpenPreNotify += this.swAppSW_CommandOpenPreNotify;
        swAppSW.CommandCloseNotify += this.swAppSW_CommandCloseNotify;
    }

    private int swAppSW_CommandOpenPreNotify(int command, int userCommand)
    {
        //Send message when the Fillet PropertyManager page is about to open
        if ((command == (int)swCommands_e.swCommands_Fillet)) MessageBox.Show("Fillet PropertyManager
page is about to open.");
        return 0;
    }

    private int swAppSW_CommandCloseNotify(int command, int reason)
    {
        //Send message when the Fillet PropertyManager page is canceled
        MessageBox.Show("Fillet PropertyManager page was canceled.");
        return 0;
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}

```

Fire Events When PropertyManager Page Opened and Canceled Example (VBA)

This example shows how to get the SOLIDWORKS command ID, PropertyManager title, and whether the user interface is active. Events are fired before the PropertyManager page is opened and when it is canceled.

```
'-----
' Preconditions:
' 1. Copy Main into the macro.
' 2. Click Insert > Class Module and copy Class1
'    into the module.
' 2. Add a reference to SOLIDWORKS <version>
'    commands type library.
' 3. Open the Immediate window.
' 4. Verify that the part to open exists.
'
' Postconditions:
' 1. Opens the part.
' 2. Fires the CommandOpenPreNotify event; click
'    OK to close the message box.
' 3. Opens the Fillet PropertyManager page.
' 4. Gets the title of the PropertyManager page, whether the
'    user interface is active, and whether the command ID
'    is a fillet.
' 5. Click X on the Fillet PropertyManager
'    page to cancel it.
' 6. Fires the CommandCloseNotify event; click OK
'    to close the message box.
' 7. Examine the Immediate window.
'-----
'Main

Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim modelName As String
Dim errors As Long
Dim warnings As Long
Dim commandID As Long
Dim pmpTitle As String
Dim isUIActive As Boolean
Dim swAppEvents As New Class1

Sub main()

Set swApp = Application.SldWorks

' Set up event
Set swAppEvents.swApp = swApp

' Open the model
modelName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\fillets\knob.sldprt"
Set swModel = swApp.OpenDoc6(modelName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)
Set swModelDocExt = swModel.Extension

' Open the Fillet PropertyManager Page,
' which causes the CommandOpenPreNotify event
' to fire
swModelDocExt.RunCommand swCommands_Fillet, "Fillet"

' Get the command ID if the command is a fillet,
' get the PropertyManager page title if one is active,
' and get whether the user interface is active
swApp.GetRunningCommandInfo commandID, pmpTitle, isUIActive
If pmpTitle <> "" Then Debug.Print "Title of PropertyManager page: " & pmpTitle
Debug.Print "Is user interface active? " & isUIActive
If (commandID = 9) Then
    Debug.Print "Command ID: " & "swCommands_Fillet"
Else
    Debug.Print "Command ID: " & "Not a fillet."
End If
```

End Sub

Class1

Option Explicit

Public WithEvents swApp As SldWorks.SldWorks

```
Private Function swApp_CommandOpenPreNotify(ByVal command As Long, ByVal userCommand As Long) As Long
    'Send message when the Fillet PropertyManager page is about to open
    If (command = swCommands_e.swCommands_Fillet) Then MsgBox "Fillet PropertyManager page is about to open."
End Function

Private Function swApp_CommandCloseNotify(ByVal command As Long, ByVal reason As Long) As Long
    'Send message because the Fillet PropertyManager page was canceled by clicking the X button
    MsgBox "Fillet PropertyManager page was canceled."
End Function
```

Fire Events When PropertyManager Page Opened and Canceled Example (VB.NET)

This example shows how to get the SOLIDWORKS command ID, PropertyManager title, and whether the user interface is active. Events are fired before the PropertyManager page is opened and when it is canceled.

```
'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Add a reference to SolidWorks.Interop.swcommands.dll.
' 3. Open the Immediate window.
' 4. Clear Tools > Options > Stop VSTA debugger on
'    macro exit, if it's selected.
'
' Postconditions:
' 1. Opens the part.
' 2. Fires the CommandOpenPreNotify event; click OK
'    to close the message box.
' 3. Opens the Fillet PropertyManager page.
' 4. Gets the title of the PropertyManager page, whether the
'    user interface is active, and whether the command ID
'    is a fillet.
' 5. Click X on the Fillet PropertyManager page
'    to cancel it.
' 6. Fires the CommandCloseNotify event; click OK to close
'    the message box.
' 7. Examine the Immediate window.
' 8. Click Stop Debugging in the IDE.
' 9. Select Tools > Options > Stop VSTA debugger on
'    macro exit, if you cleared it in Preconditions
'    step 4.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
Imports SolidWorks.Interop.swcommands
```

```
Partial Class SolidWorksMacro
```

```
    Public WithEvents swAppSW As SldWorks
```

```
    Public Sub Main()
```

```
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim modelName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim commandID As Integer
        Dim pmpTitle As String = ""
        Dim isUIActive As Boolean
```

```
        ' Set up event
        swAppSW = swApp
        AttachEventHandlers()
```

```
        ' Open the model
        modelName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\fillets\knob.sldprt"
```

```
        swModel = swApp.OpenDoc6(modelName, swDocumentTypes_e.swDocPART,
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
```

```
        ' Open the Fillet PropertyManager Page,
        ' which causes the CommandOpenPreNotify event
        ' to fire
```

```
        swModelDocExt.RunCommand(swCommands_e.swCommands_Fillet, "Fillet")
```

```

' Get the command ID if the command is a fillet,
' get the PropertyManager page title if one is active,
' and get whether the user interface is active
swApp.GetRunningCommandInfo(commandID, pmpTitle, isUIActive)
If pmpTitle <> "" Then Debug.Print("Title of PropertyManager page: " & pmpTitle)
Debug.Print("Is user interface active? " & isUIActive)
If (commandID = 9) Then
    Debug.Print("Command ID: " & "swCommands_Fillet")
Else
    Debug.Print("Command ID: " & "Not a fillet.")
End If

End Sub

Sub AttachEventHandlers()
    AttachSWEvents()
End Sub
Sub AttachSWEvents()
    AddHandler swAppSW.CommandOpenPreNotify, AddressOf Me.swAppSW_CommandOpenPreNotify
    AddHandler swAppSW.CommandCloseNotify, AddressOf Me.swAppSW_CommandCloseNotify
End Sub
Private Function swAppSW_CommandOpenPreNotify(ByVal command As Integer, ByVal userCommand As Integer)
As Integer
    'Send message when the Fillet PropertyManager page is about to open
    If (command = swCommands_e.swCommands_Fillet) Then MsgBox("Fillet PropertyManager page is about
to open.")
End Function
Private Function swAppSW_CommandCloseNotify(ByVal command As Integer, ByVal reason As Integer) As
Integer
    'Send message when the Fillet PropertyManager page was canceled by clicking the X button
    MsgBox("Fillet PropertyManager page was canceled.")
End Function

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Fire Notification After Adding a Mate Example (C#)

This example shows how to fire a post-notify event when adding mates.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly to open exists.
// 2. Click Tools > Options > clear the Stop VSTA debugger
//    on macro exit check box if it is selected.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Opens the assembly.
// 2. Interactively add a mate between two entities
//    (Click Insert > Mate). For example, add a lock mate
//    between the toaster base and the cup.
// 3. Adds a mate between the selected entities.
// 4. Examine the Immediate window.
// 5. Click Debug > Stop Debugging in SOLIDWORKS Visual
//    Studio Tools for Applications.
// 6. Click Tools > Options > Stop VSTA debugger on macro exit
//    check box in SOLIDWORKS.
//
// NOTE: Because this assembly is used elsewhere, do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Collections;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public AssemblyDoc swAssem;

        public void Main()
        {
            ModelDoc2 swModel;
            int errors = 0;
            int warnings = 0;
            Hashtable openAssem;

            swModel = (ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2019\\samples\\tutorial\\api\\toaster_scene.sldasm", (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpe
nDocOptions_Silent, "", ref errors, ref warnings);

            //Set up events
            swAssem = (AssemblyDoc)swModel;
            openAssem = new Hashtable();
            AttachEventHandlers();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swAssem.AddMatePostNotify2 += this.swAssem_AddMatePostNotify2;
        }

        private int swAssem_AddMatePostNotify2(ref object mates)
        {
            Debug.Print("One or more mates were added to the assembly.");
            return 1;
        }
    }
}
```

Fire Notification After Adding a Mate Example (VBA)

This example shows how to fire a post-notify event when adding mates:

```
'-----
' Preconditions:
' 1. Verify that the specified assembly to open exists.
' 2. Copy Main to the main module.
' 3. Click Insert > Class Module and copy Class to that
'    module.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Opens the assembly.
' 2. Interactively add a mate between two entities
'    (click Insert > Mate). For example, add a lock mate
'    between the toaster base and the cup.
' 3. Adds a mate between the selected entities.
' 4. Examine the Immediate window.
'
' NOTE: Because this assembly is used elsewhere, do not save
' changes.
'-----
'Main
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssemblyDoc As SldWorks.AssemblyDoc
Dim errorstatus As Long, warningstatus As Long
Dim swAssemblyDocEvents As Class1

Sub main()

    Set swApp = Application.SldWorks
    swApp.OpenDoc6 "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\toaster_scene.sldasm",
swDocASSEMBLY, swOpenDocOptions_Silent, "", errorstatus, warningstatus
    Set swModel = swApp.ActiveDoc
    Set swAssemblyDoc = swModel

    ' Set up event
    Set swAssemblyDocEvents = New Class1
    Set swAssemblyDocEvents.swAssemblyDoc = swApp.ActiveDoc

End Sub

'Class

Option Explicit

Public WithEvents swAssemblyDoc As SldWorks.AssemblyDoc

Private Function swAssemblyDoc_AddMatePostNotify2(mates As Variant) As Long
    Debug.Print "One or more mates were added to the assembly."

    Dim mate As Mate2
    Dim i As Integer
    For i = 0 To UBound(mates)
        Set mate = mates(i)
        Debug.Print "Type of mate added as defined in swMateType_e: " & mate.Type
    Next i

End Function
```